

Machine Programming

Lecture 10 – Agentic Frameworks: Selected Topics

Ziyang Li

Logistics – Week 5

- Assignment 2

Assignment 2 - Evaluating Coding LLMs

SEP 25, 2026 6:08 AM

OCT 9, 2026 11:59 PM

4

100%

☐

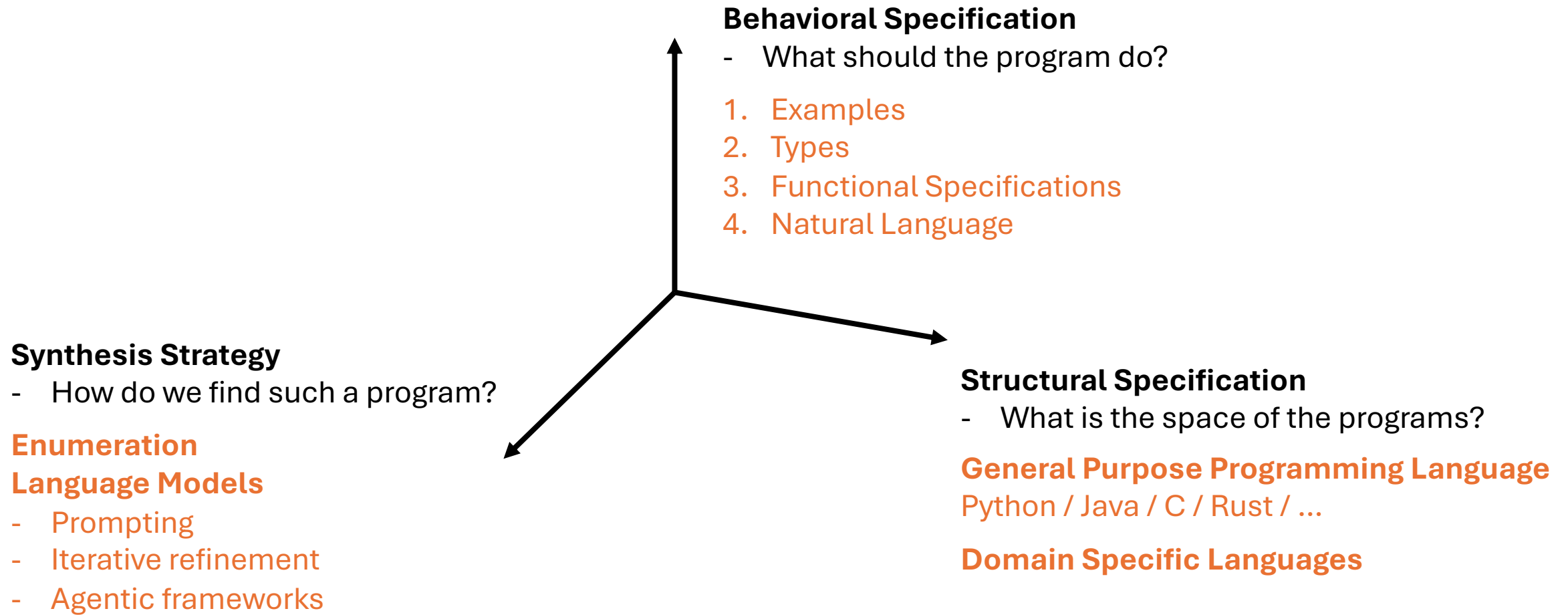
ON

⋮

Late Due Date: OCT 12, 2026 11:59 PM

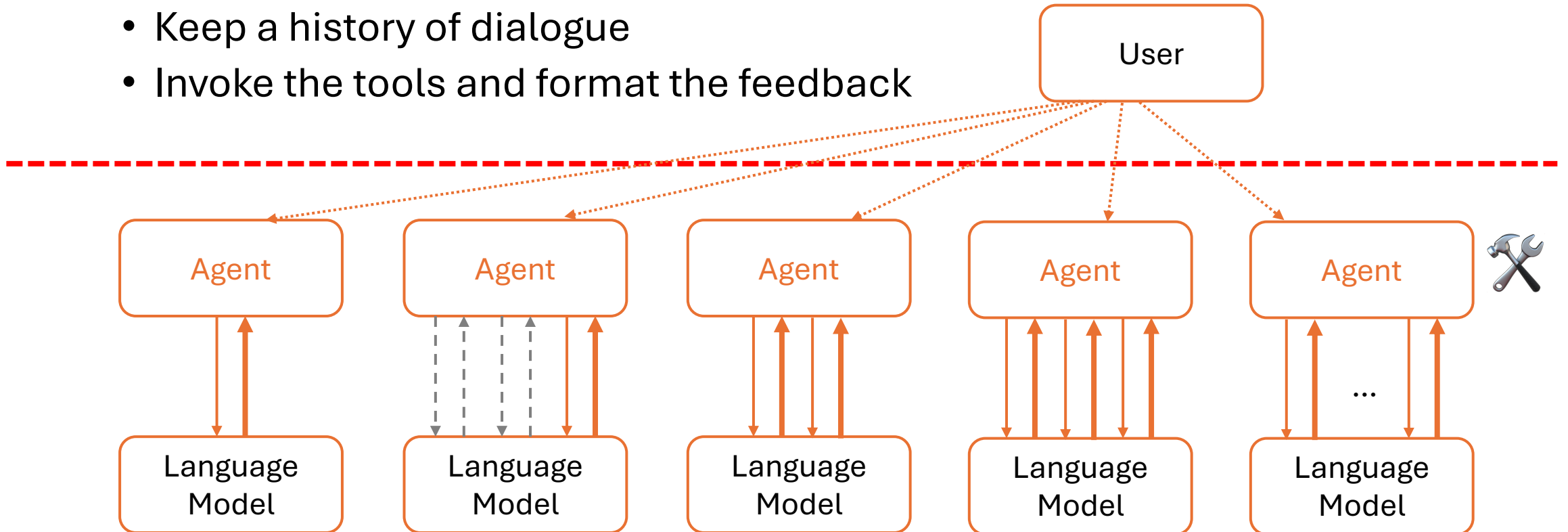
- Due in two weeks (Oct 9) since release last Friday
- In case your LLM API key is running out of credit, please register for your own key following our tutorial!

The Course So Far

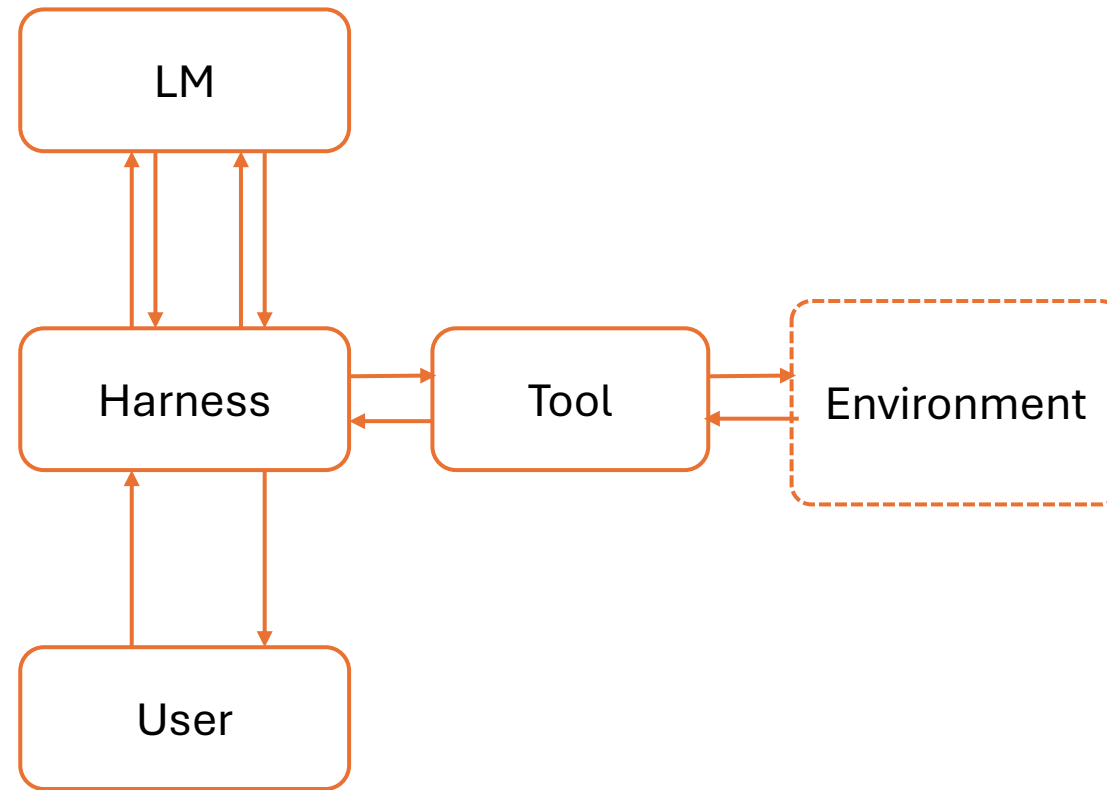


Agentic Pipelines

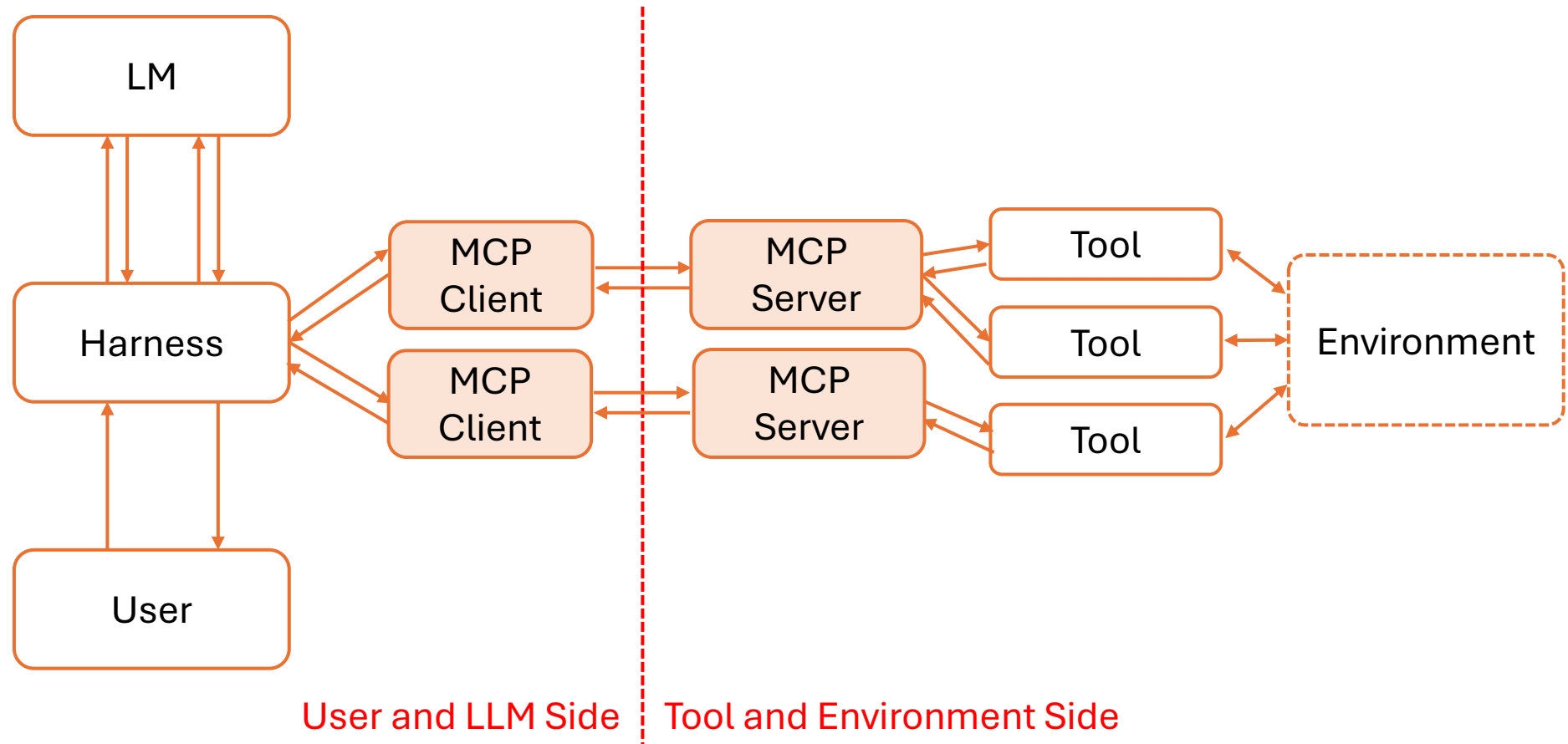
- **Agent** holds the responsibility to
 - Send requests to LLM
 - Keep a history of dialogue
 - Invoke the tools and format the feedback

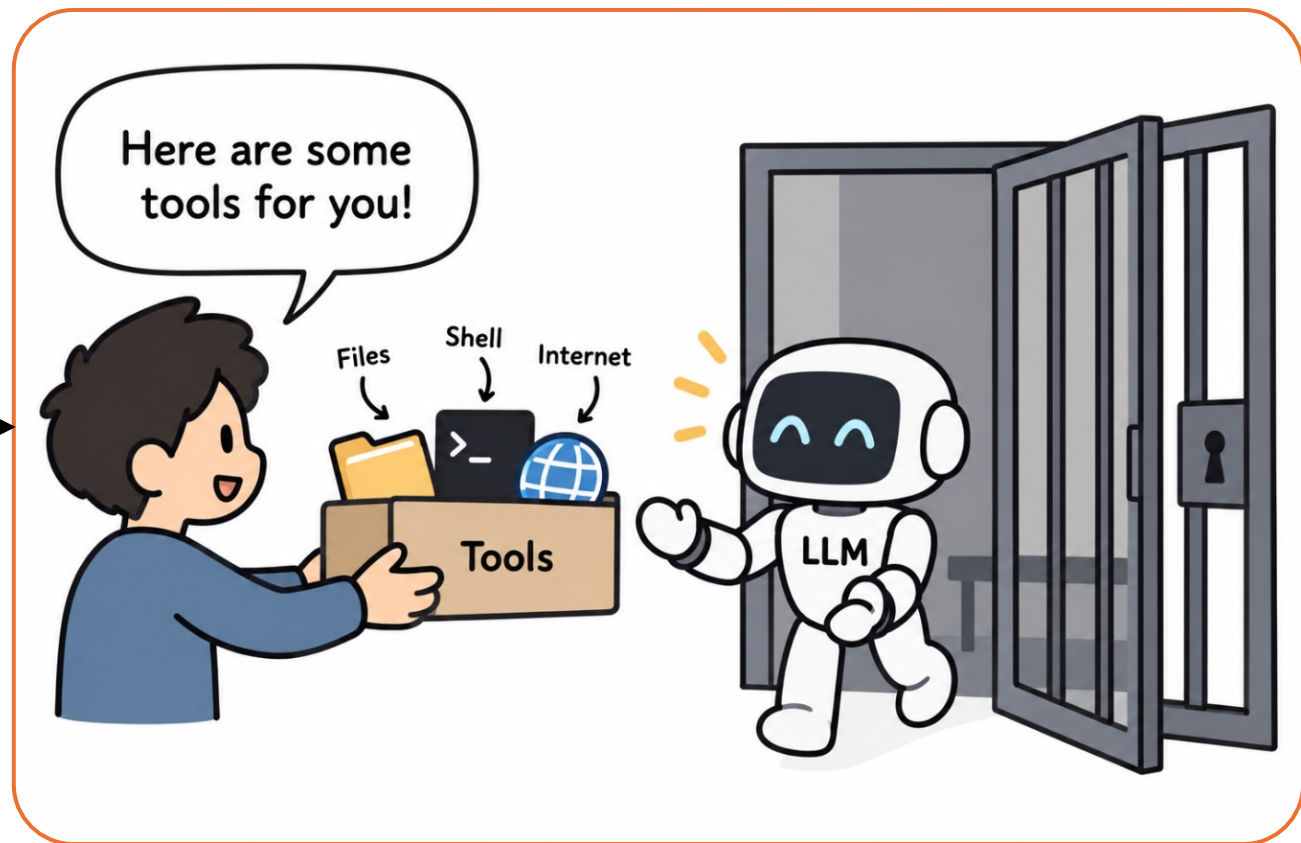
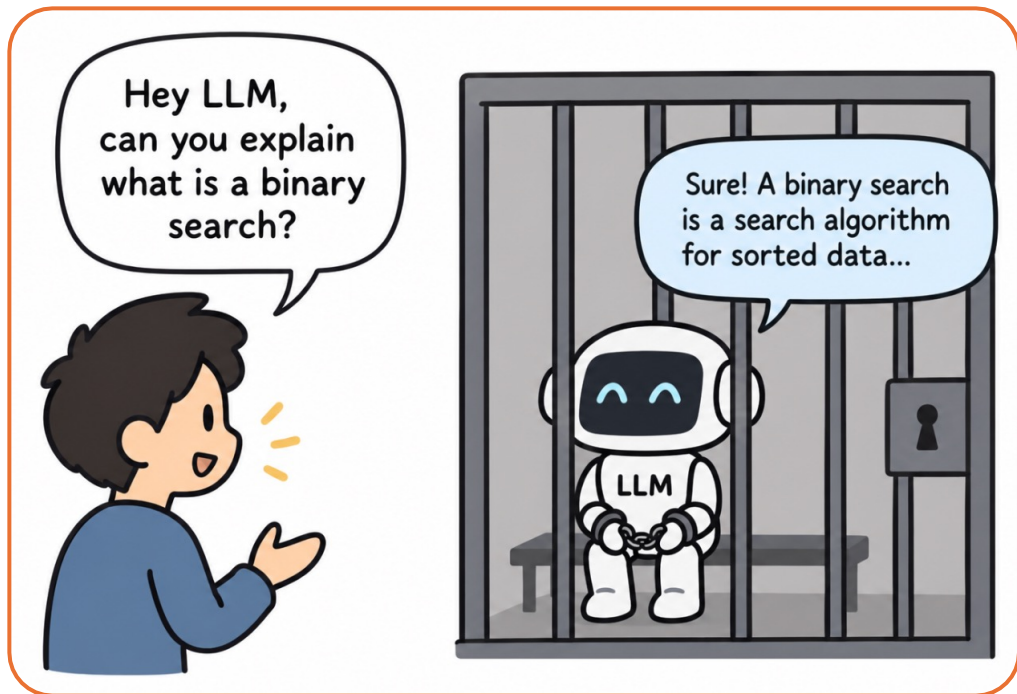


Agentic Framework



Model Context Protocol (MCP)

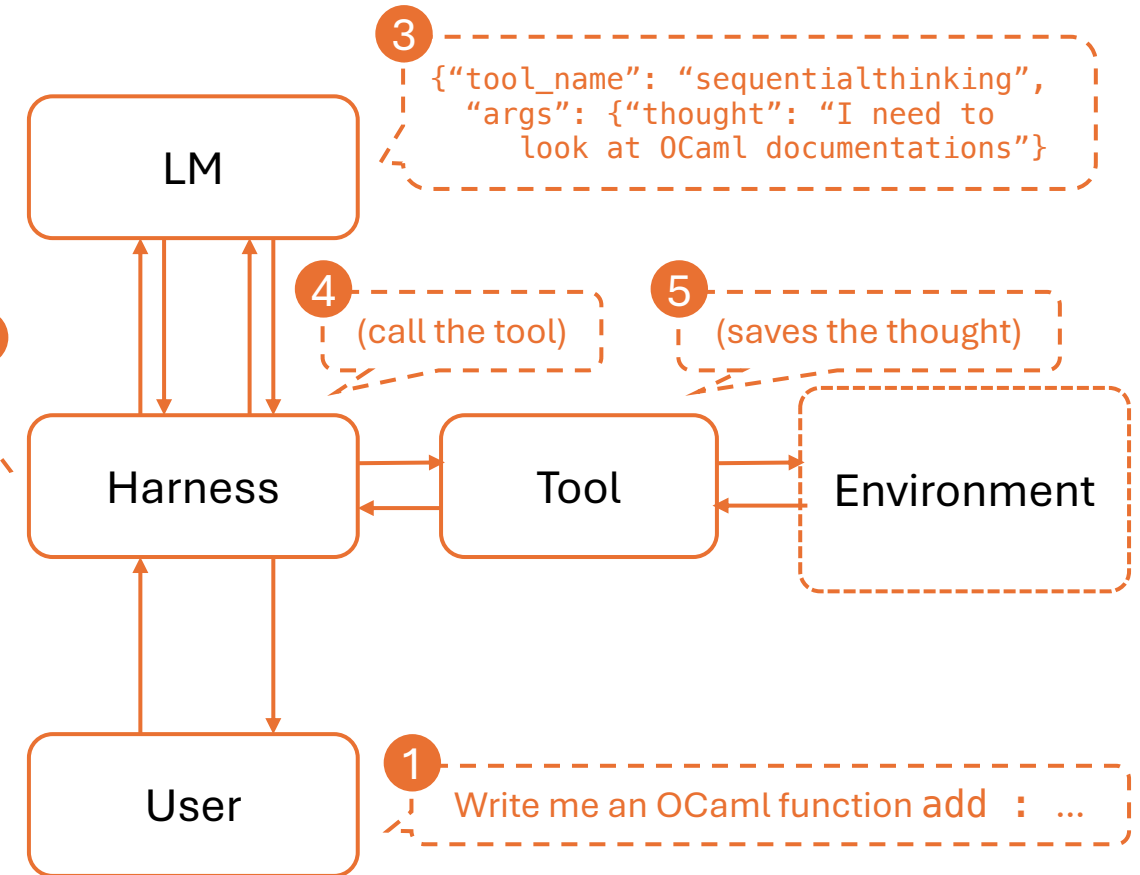




Sequential Thinking Tool

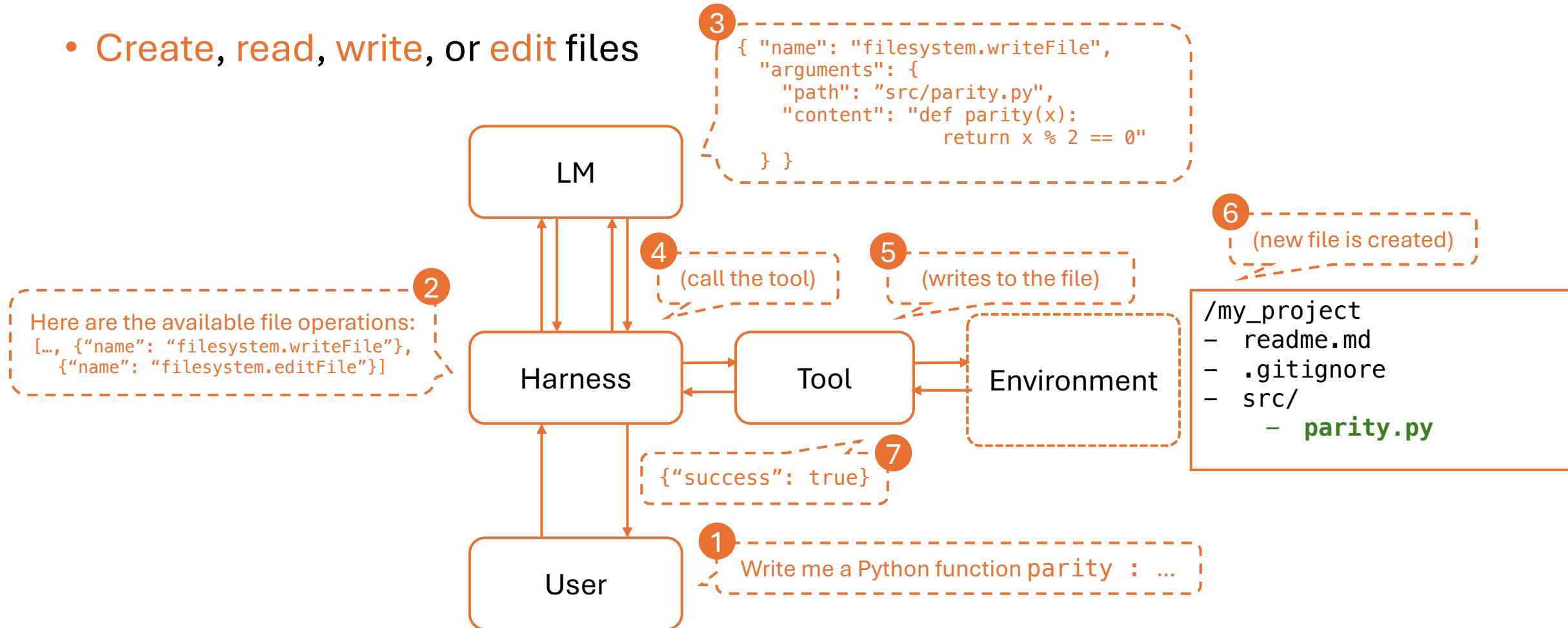
- **thought** (string)
 - the current step you want to record
- **nextThoughtNeeded** (boolean)
 - whether you inter
- **thoughtNumber** (in
 - index of this step
- **totalThoughts** (int ≥ 1)
 - your *current* plan for how many steps you'll need

The user wants to write a function in OCaml. Here are the available tools: [..., {"name": "sequentialthinking"}]

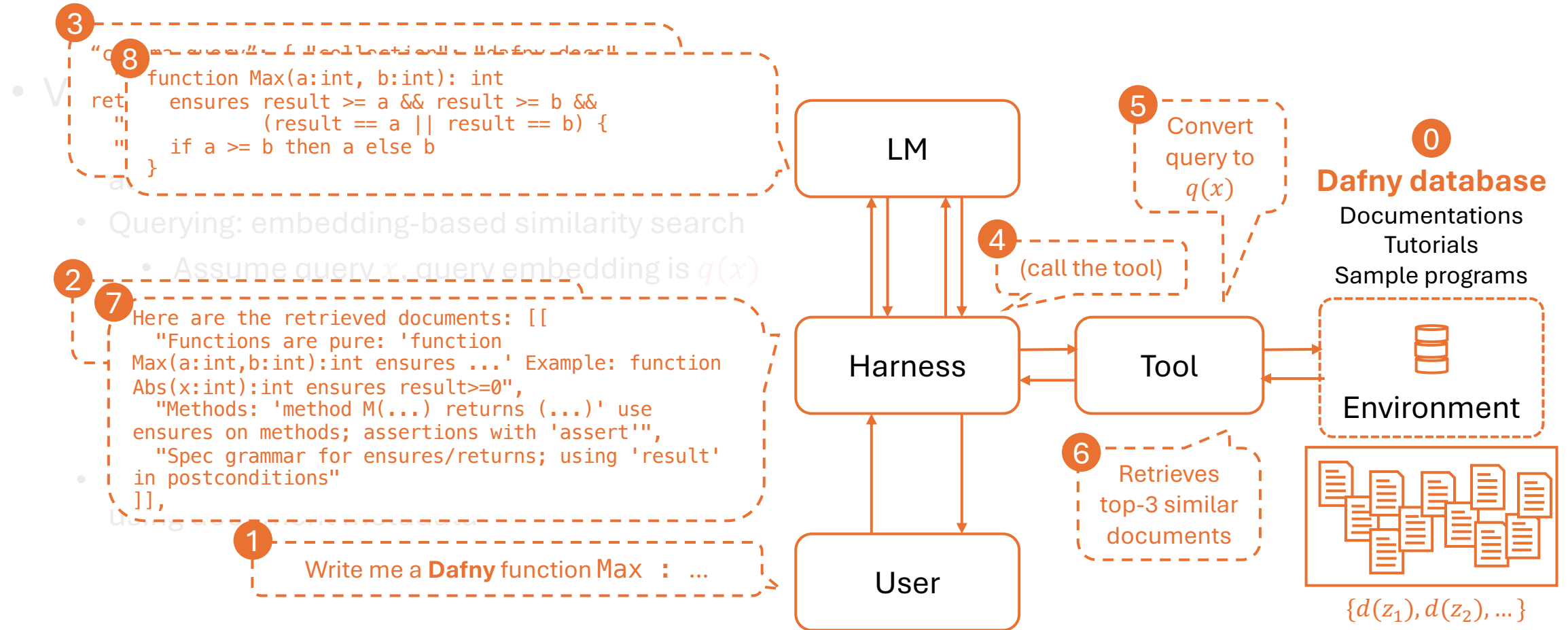


File System (FS) Tool

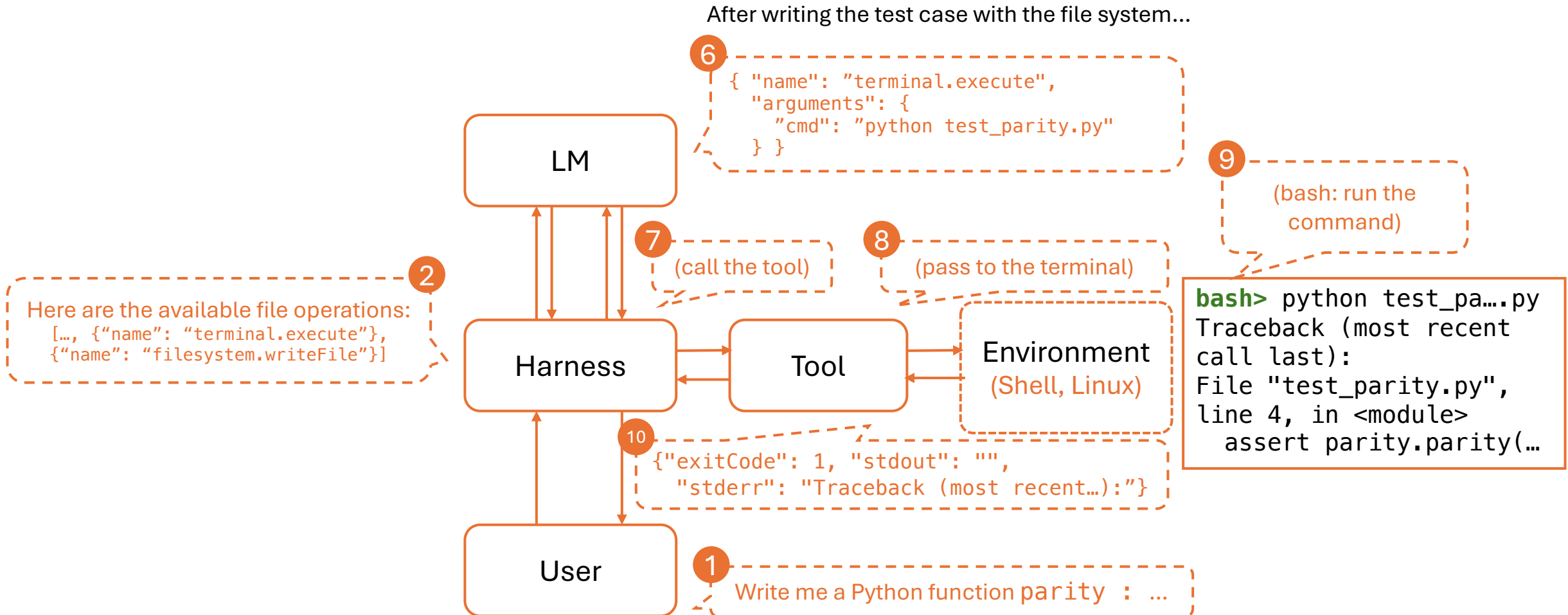
- Create, read, write, or edit files



RAG Database as a Tool



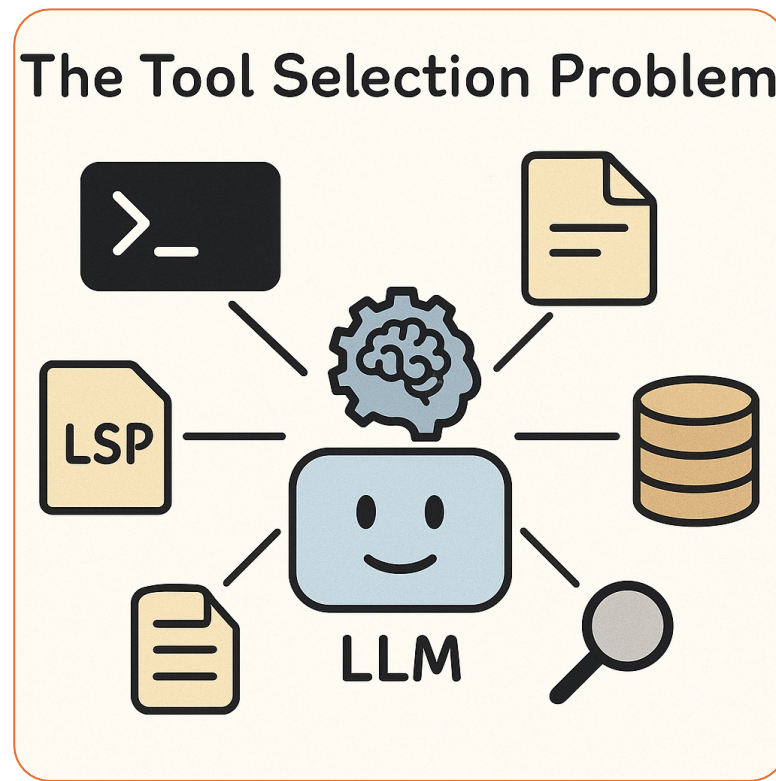
Terminal as a Tool



Topics of Today

- Selected topics of agentic systems
 - Tool selection problem
 - Context management
 - Grammar-constrained decoding
 - Skills
 - Harness engineering

Tool Selection Problem



Tool Selection Problem

- Agents often have **too many tools** available:
 - Sequential thinking, web search, file system, vector database, terminal, language servers, CI/CD pipelines, reading PDFs, looking at images, etc.
 - Auxiliary tools: get date/time, user info, local IDE context, system info
- Wrong tool choice leads to...
 - Wasted tokens, cost, wrong fixes, security loopholes...
- How could LLM agent plan and know what tool to use?
 - Challenge 1: multiple ways to solve the same problem
 - Challenge 2: some are costly but precise, others are cheap but shallow
 - Challenge 3: error attribution – which tool call led to a failure outcome?

Anatomy of One Real Tool Definition

OpenHands terminal tool, as serialized for the OpenAI function API. 883 tokens total.

```
{"type":"function","function":{"name": "terminal",  
  "description": "Execute a shell command in the terminal within a  
    persistent shell session. ### Command Execution * One command at  
    a time... * Do NOT use `set -e`... ### Long-running Commands ..."  
  "parameters": {"type":"object","properties": {  
    "command": {type, 520-char description} 155 tokens  
    "timeout": {type, 443-char description} 104 tokens  
    "reset": {type, 238-char description} 53 tokens  
    "summary": {type, 178-char description} 43 tokens  
    "is_input": {type, 133-char description} 36 tokens  
  }, "required": ["command"]}}}
```

The Measurements

Harness / configuration	Tools	Tokens	% of 200K
mini-SWE-agent, text mode (default)	0	0	0.00%
mini-SWE-agent, tool-call mode	1	47	0.02%
Codex CLI, core tools only, no MCP	10	2,973	1.49%
Codex CLI, + 9 MCP tools, shown in full	15	6,441	3.22%
Codex CLI, + same 9 MCP tools, deferred	6	1,072	0.54%
OpenHands, cli_mode preset	5	3,569	1.78%
OpenHands, default preset (+browser)	19	5,943	2.97%
Claude Code, the 8 tools loaded up front	8	9,418	4.71%
Claude Code, 57 more tools, names only	57	681	0.34%

Defer the Schemas, Keep the Names

EAGER (one trip)

```
turn 1  all 65 schemas .. 21K-103K*  
model: create_event(...)
```

DEFERRED (two trips)

```
turn 1  turn 1  8 schemas + 57 names ... 10.1K  
model: ToolSearch("calendar")
```

```
turn 1  + create_event schema .... 1.6K  
model: create_event(...)
```

On the Tool Manipulation Capability of Open-source Large Language Models

Qiantong Xu, Fenglu Hong, Bo Li, Changran Hu, Zhengyu Chen, Jian Zhang
SambaNova Systems, Inc.
Palo Alto, CA, USA
`{qiantong.xu,jian.zhang}@sambanovasystems.com`

On the Tool Manipulation Capability of Open-source Large Language Models

Qiantong Xu, Feng

{qiantong

Table 4: Tasks in the ToolBench. We provide demonstration examples for few-shot in-context-learning while test cases are for quantitatively evaluation. We develop API complexity, a metric to quantify the challenge level in generalizing to unseen API combinations; higher complexity indicates more challenging tasks. We package the challenges beyond API complexity as advanced reasoning. We refer to Appendix A for more details on these tasks.

Task	Single Step						Multi-Step	
	Open Weather	The Cat API	Home Search	Trip Booking	Google Sheets	VirtualHome	WebShop Long / Short	Tabletop
<i>Data</i>								
API functions	9	6	15	20	108	40	2	32
Demonstration examples	18	12	10	11	10	83	1533 / 200	74
Test cases	100	100	100	120	70	100	100	105
<i>Level of challenges</i>								
API complexity	2.2	1.4	7.3	11.1	8.4	12.3	0.0	4.6
Advanced reasoning					✓		✓	✓

On the Tool Manipulation Capability of

Table 4: Tasks in the ToolBench. We provide demonstration examples for few-shot in-context-learning while test cases are for quantitatively evaluation. We develop API complexity, a metric to quantify the challenge level in generalizing to unseen API combinations; higher complexity indicates more challenging tasks. We package the challenges beyond API complexity as advanced reasoning. We refer to Appendix A for more details on these tasks.

Task
<i>Data</i>
API functions
Demonstration example
Test cases
<i>Level of challenges</i>
API complexity
Advanced reasoning

Table 6: Capability gap in tool manipulation is substantial between closed API and open-source LLMs in the out-of-the-box zero-shot setting. Using model alignment, the in-context demonstration retriever and the system prompt, open-sourced LLMs attain significant boost in success rate. GPT-4 is enhanced with the retriever and system prompt. Tabletop is only evaluated in the few-shot fashion.

Task	Open Weather	The Cat API	Home Search	Trip Booking	Google Sheets	VirtualHome	WebShop Long	WebShop Short	Tabletop
<i>Zero-shot Baseline</i>									
GPT-4	81.3	97.4	76.6	91.5	5.7	40.8 / 8.0		0.0	-
LLaMA-30b	39.0	49.0	0.0	0.0	0.0	78.0 / 0.3		0.0	-
StarCoder	32.0	71.0	7.0	13.3	5.9	22.0 / 3.7		0.0	-
CodeGen-16B-mono	7.0	78.0	0.0	0.0	1.4	4.0/ 1.0		0.0	-
<i>Enhanced w/ techniques</i>									
GPT-4	99.0	98.0	98.0	99.2	68.6	29.0 / 21.7	0.0	0.0	83.8
LLaMA-30b	100.0	94.0	87.0	85.8	2.9	16.0 / 24.3	0.0	0.0	7.5
StarCoder	99.0	97.0	83.0	80.8	21.2	31.0 / 18.4	0.0	0.0	13.9
CodeGen-16B-mono	97.7	99.0	82.0	77.5	19.8	29.0 / 17.2	0.0	3.5	16.2

TOOLGEN: UNIFIED TOOL RETRIEVAL AND CALLING VIA GENERATION

Renxi Wang^{1,2}

Xudong Han^{1,2}

Lei Ji³

Shu Wang⁴

Timothy Baldwin^{1,2,5}

Haonan Li^{1,2}

¹LibrAI

²Mohamed bin Zayed University of Artificial Intelligence

³Microsoft

⁴University of California, Los Angeles

⁵The University of Melbourne

{renxi.wang, xudong.han, timothy.baldwin, haonan.li}@mbzuai.ac.ae

leiji@microsoft.com

shuwang0712@ucla.edu

TOOLGEN: UNIFIED TOOL RETRIEVAL AND CALLING VIA GENERATION

Renxi Wang^{1,2}

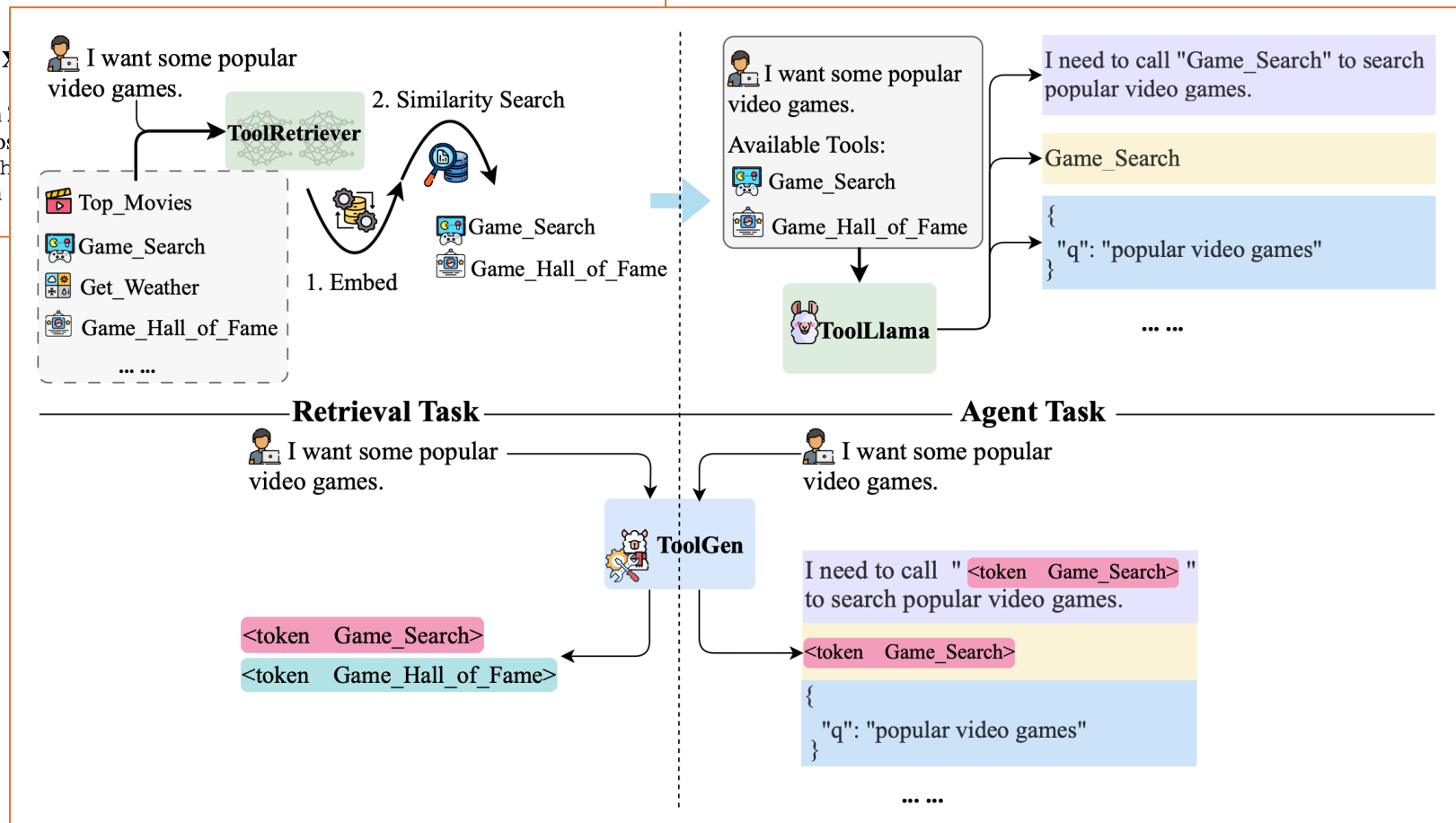
Timothy Baldwin^{1,2,5}

¹LibrAI ²Mohamed bin

⁴University of California, Los Angeles

{renxi.wang, xudong.hu}

lei@ucla.edu, lei@microsoft.com



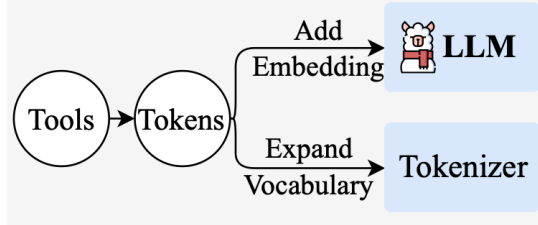
TOOLGEN: UNIFIED TOOL RETRIEVAL AND CALLING

VIA G

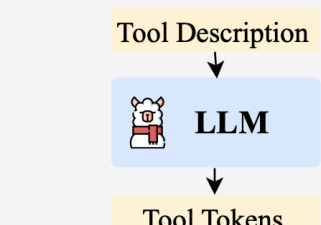
Renxi Wang
Timothy
LibrAI
Universit
{renxi.
lei@



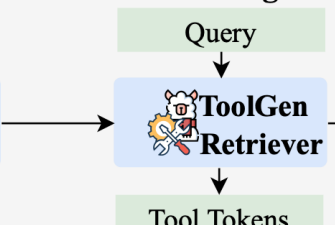
Tool Virtualization



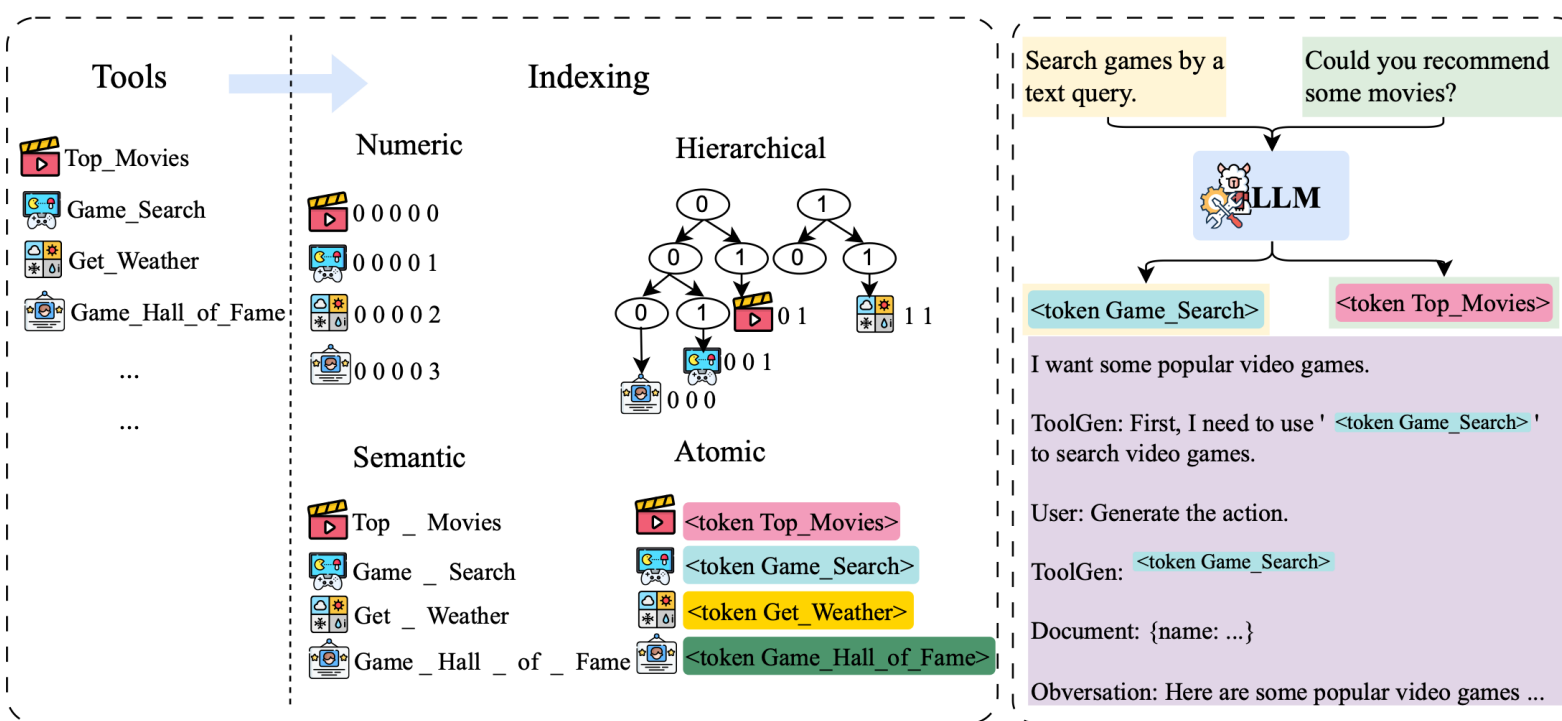
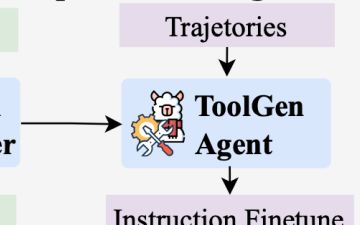
Tool Memorization



Retrieval Training



Pipeline Training



TOOLGEN: UNIFIED TOOL RETRIEVAL AND CALLING

VIA G

Renxi Wang
Timothy
LibrAI
Universit
{renxi.
lei@

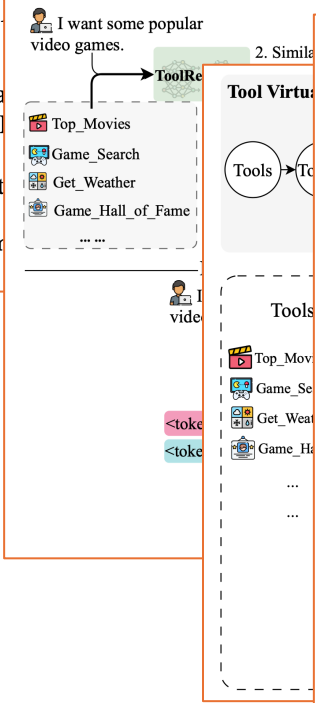


Table 1: Tool retrieval evaluation across two settings: (1) **In-Domain**, where models are trained and evaluated within the same domain; and (2) **Multi-Domain**, where models are trained on all domains and evaluated with the full set of tools across all domains. BM25, EmbSim, and Re-Invoke are unsupervised baselines without training. IterFeedback is retrieval system with multiple models and feedback mechanism. ToolRetriever is trained using contrastive learning, while ToolGen is trained with next-token prediction. Results marked with * were not implemented by us and are copied from their original paper, and hence only in the In-Domain setting. For ToolGen in the In-Domain setting, we allow the generation space to include all tokens, which is a more challenging scenario compared to other models. Best results in each category are **bolded**.

Model	I1			I2			I3		
	NDCG1	NDCG3	NDCG5	NDCG1	NDCG3	NDCG5	NDCG1	NDCG3	NDCG5
In-Domain									
BM25	29.46	31.12	33.27	24.13	25.29	27.65	32.00	25.88	29.78
Long-Context LLM*	32.22	42.87	52.14	25.39	33.91	46.07	25.11	32.57	44.03
EmbSim	63.67	61.03	65.37	49.11	42.27	46.56	53.00	46.40	52.73
Re-Invoke*	69.47	–	61.10	54.56	–	53.79	59.65	–	59.55
IterFeedback*	90.70	90.95	<u>92.47</u>	<u>89.01</u>	<u>85.46</u>	<u>87.10</u>	91.74	87.94	90.20
ToolRetriever	80.50	79.55	84.39	71.18	64.81	70.35	70.00	60.44	64.70
ToolGen	<u>89.17</u>	<u>90.85</u>	92.67	91.45	88.79	91.13	<u>87.00</u>	<u>85.59</u>	<u>90.16</u>
Multi-Domain									
BM25	22.77	22.64	25.61	18.29	20.74	22.18	10.00	10.08	12.33
EmbSim	54.00	50.82	55.86	40.84	36.67	39.55	18.00	17.77	20.70
ToolRetriever	72.31	70.30	74.99	64.54	57.91	63.61	52.00	39.89	42.92
ToolGen	87.67	88.84	91.54	83.46	86.24	88.84	79.00	79.80	84.79

Note on NDCG Metrics

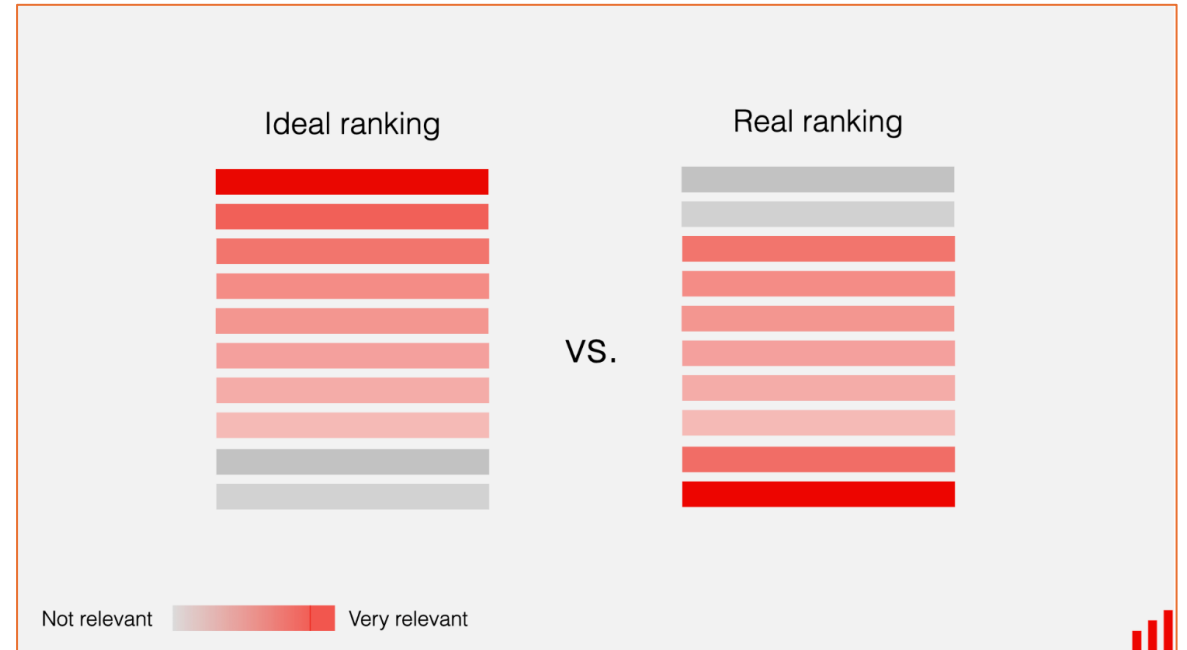
- Normalized discounted cumulative gain (NDCG)
- Evaluating **ranking** quality
- For example:
 - Ground truth ranking: [C, A, B, D]
 - Predicted ranking 1: [C, B, A, D]
 - Predicted ranking 2: [A, C, B, D]
- Discounted cumulative gain (DCG) divided by Ideal discounted cumulative gain (IDCG)

Table 1: Tool retrieval evaluation across two settings: (1) **In-Domain**, where models are trained and evaluated within the same domain; and (2) **Multi-Domain**, where models are trained on all domains and evaluated with the full set of tools across all domains. BM25, EmbSim, and Re-Invoke are unsupervised baselines without training. IterFeedback is retrieval system with multiple models and feedback mechanism. ToolRetriever is trained using contrastive learning, while ToolGen is trained with next-token prediction. Results marked with * were not implemented by us and are copied from their original paper, and hence only in the In-Domain setting. For ToolGen in the In-Domain setting, we allow the generation space to include all tokens, which is a more challenging scenario compared to other models. Best results in each category are **bolded**.

Model	I1			I2			I3		
	NDCG1	NDCG3	NDCG5	NDCG1	NDCG3	NDCG5	NDCG1	NDCG3	NDCG5
In-Domain									
BM25	29.46	31.12	33.27	24.13	25.29	27.65	32.00	25.88	29.78
Long-Context LLM*	32.22	42.87	52.14	25.39	33.91	46.07	25.11	32.57	44.03
EmbSim	63.67	61.03	65.37	49.11	42.27	46.56	53.00	46.40	52.73
Re-Invoke*	69.47	–	61.10	54.56	–	53.79	59.65	–	59.55
IterFeedback*	90.70	90.95	<u>92.47</u>	<u>89.01</u>	<u>85.46</u>	<u>87.10</u>	91.74	87.94	90.20
ToolRetriever	80.50	79.55	84.39	71.18	64.81	70.35	70.00	60.44	64.70
ToolGen	<u>89.17</u>	<u>90.85</u>	92.67	91.45	88.79	91.13	<u>87.00</u>	<u>85.59</u>	<u>90.16</u>
Multi-Domain									
BM25	22.77	22.64	25.61	18.29	20.74	22.18	10.00	10.08	12.33
EmbSim	54.00	50.82	55.86	40.84	36.67	39.55	18.00	17.77	20.70
ToolRetriever	72.31	70.30	74.99	64.54	57.91	63.61	52.00	39.89	42.92
ToolGen	87.67	88.84	91.54	83.46	86.24	88.84	79.00	79.80	84.79

Note on NDCG Metrics

- Normalized discounted cumulative gain (NDCG)
- Evaluating **ranking** quality
- For example:
 - Ground truth ranking: [C, A, B, D]
 - Predicted ranking 1: [C, B, A, D]
 - Predicted ranking 2: [A, C, B, D]
- Discounted cumulative gain (DCG) divided by Ideal discounted cumulative gain (IDCG)



Note on NDCG@k Metrics

- Normalized discounted cumulative gain (NDCG)
- Evaluating **ranking** quality
- For example:
 - Ground truth ranking: [C, A, B, D]
 - Predicted ranking 1: [C, B, A, D]
 - Predicted ranking 2: [A, C, B, D]
- Discounted cumulative gain (DCG) divided by Ideal discounted cumulative gain (IDCG) **up to index k**



Topics of Today

- Selected topics of agentic systems
 - ~~Tool selection problem~~
 - Context management
 - Grammar-constrained decoding
 - Skills
 - Harness engineering

Context Management Problem

- Phenomena related to long contexts:
 - When context is too long, LLM performance starts to degrade
 - Information buried in the middle would be more likely ignored by LLM

Context Management Problem

- Phenomena related to long contexts:
 - When context is too long, LLM performance starts to degrade
 - Information buried in the middle would be more likely ignored by LLM

LongICLBench: Long-context LLMs Struggle with Long In-context Learning

♠♣ Tianle Li, ♠♣ Ge Zhang, ♠ Quy Duc Do, † Xiang Yue, ♠♣ Wenhui Chen

♠ University of Waterloo

† Carnegie Mellon University

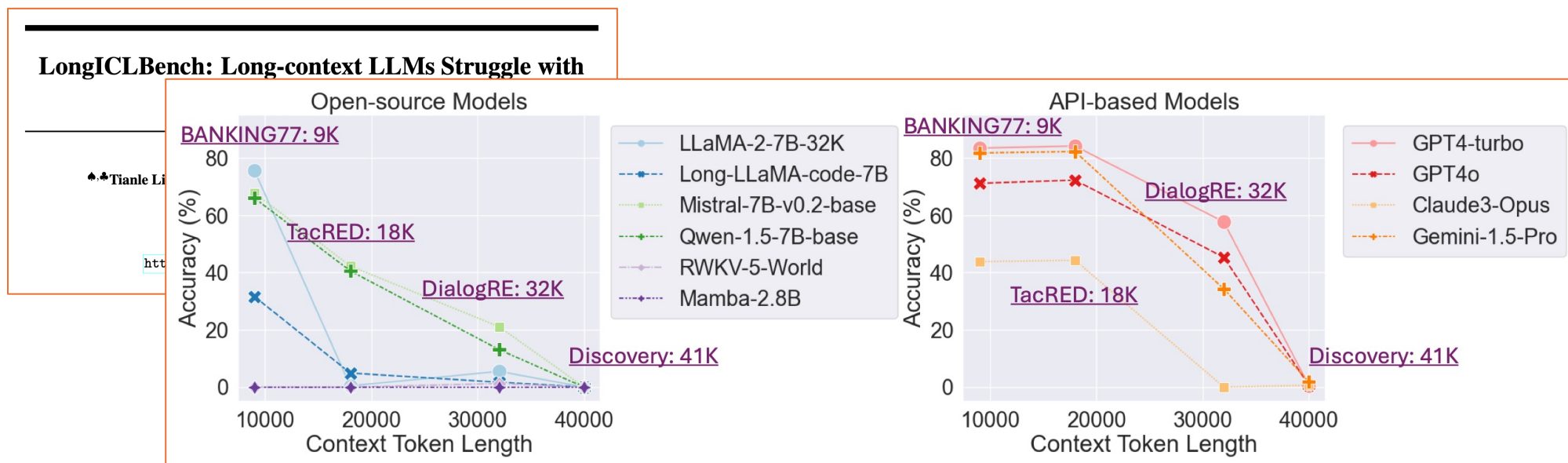
♣ Vector Institute, Toronto

{t291i, wenhuchen}@uwaterloo.ca

<https://github.com/TIGER-AI-Lab/LongICLBench>

Context Management Problem

- Phenomena related to long contexts:
 - When context is too long, LLM performance starts to degrade
 - Information buried in the middle would be more likely ignored by LLM



Context Management Problem

- Phenomena related to long contexts:
 - When context is too long, LLM performance starts to degrade
 - Information buried in the middle would be more likely ignored by LLM

Context Management Problem

- Phenomena related to long contexts:
 - When context is too long, LLM performance starts to degrade
 - Information buried in the middle would be more likely ignored by LLM

Lost in the Middle: How Language Models Use Long Contexts

Nelson F. Liu^{1*} **Kevin Lin²** **John Hewitt¹** **Ashwin Paranjape³**
Michele Bevilacqua³ **Fabio Petroni³** **Percy Liang¹**

¹Stanford University

²University of California, Berkeley

³Samaya AI

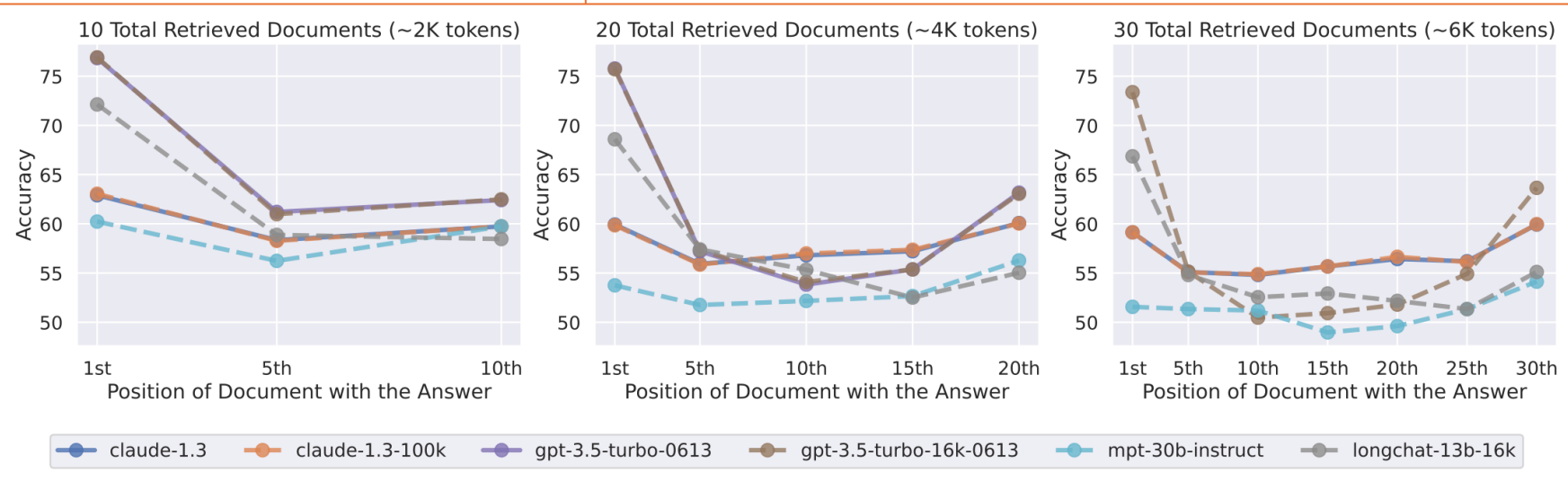
nfliu@cs.stanford.edu

Context Management Problem

- Phenomena related to long contexts:
 - When context is too long, LLM performance starts to degrade
 - Information buried in the middle would be more likely ignored by LLM

Lost in the Middle: How Language Models Use Long Contexts

Nelson F. Liu^{1*}
Michael
¹Stanford University



Context Rot

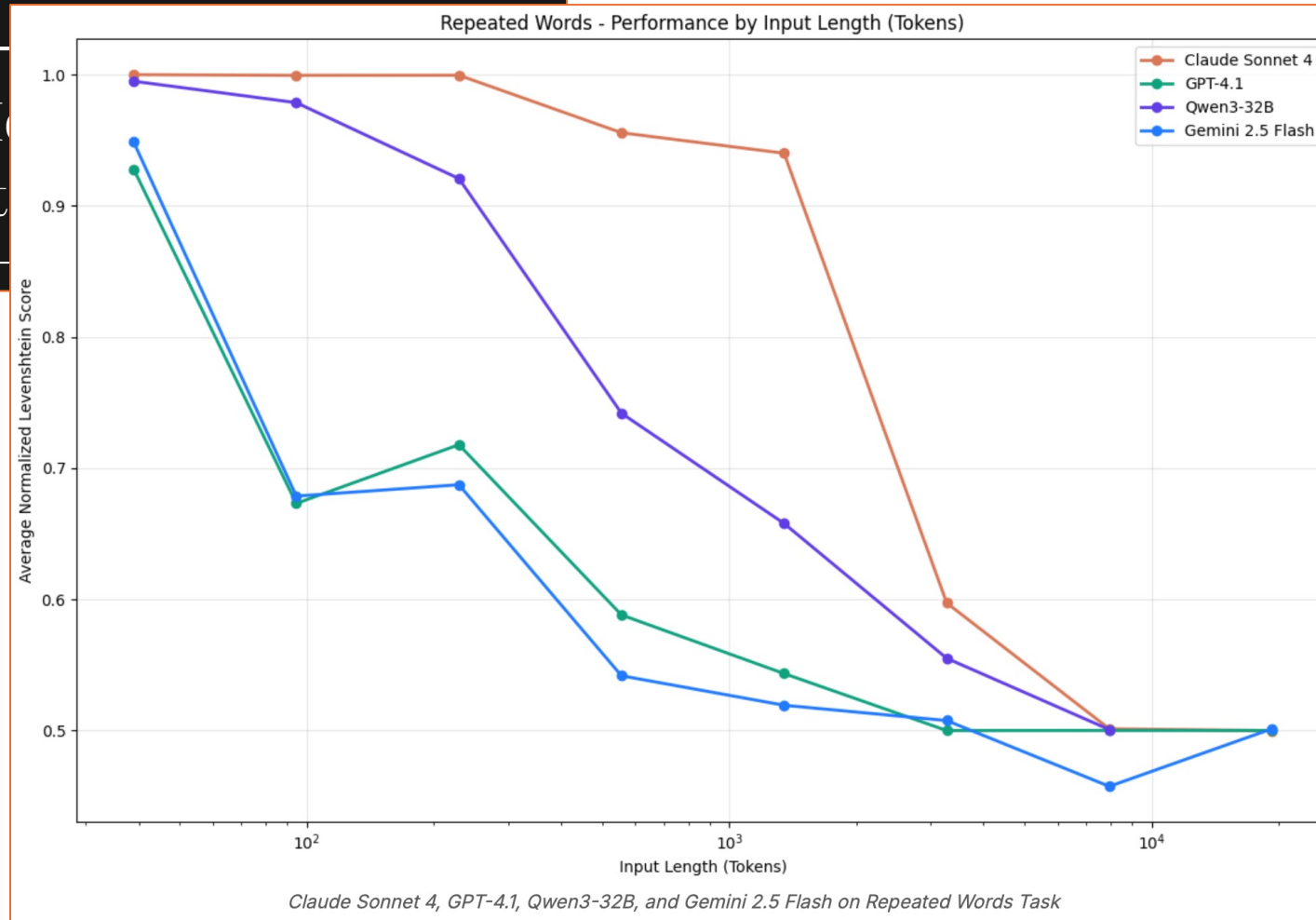


CHROMA TECHNICAL REPORT

July 14, 2025

Context Rot: How Increasing Input Tokens Impacts LLM Performance

Context Rot: How Increasing Input Tokens Impacts LLM Performance



Context Management Problem

- Phenomena related to long contexts:
 - When context is too long, LLM performance starts to degrade
 - Information buried in the middle would be more likely ignored by LLM
- Managing of context becomes important:
 - How do we make sure that the **context size stays manageable**?
 - How do we make sure that **relevant information are recognizable** by LLM?

Context Management Problem

- Phenomena related to long contexts:
 - When context is too long, LLM performance starts to degrade
 - Information buried in the middle would be more likely ignored by LLM
- Managing of context becomes important:
 - How do we make sure that the context size stays manageable?
 - How do we make sure that relevant information are recognizable by LLM?
- Solution: **Improving token efficiency of tools**
 - For costly tool calls, optimize the tokens; prefer dense information
 - Avoid showing full terminal error message logs or entire files

Summarization & Compression

- ... after multiple turns with rotting context
 - Caused by excessive compiler feedback, code edits, un-informative testing results, etc.
- Agentic framework:
 - **Summarizes** the current context...
 - **Saves** the summarization into a file...
 - **Stores** the file into RAG database...
 - **Clears** the context...
 - Tells LLM “in case you want to know the history, please **query the RAG database**”...
 - Continuing the implementation...



Active Context Management

SCULPTOR: EMPOWERING LLMs WITH COGNITIVE AGENCY VIA ACTIVE CONTEXT MANAGEMENT

Mo Li¹, L.H. Xu², Qitai Tan¹, Long Ma³, Ting Cao^{1*}, Yunxin Liu¹

¹ Tsinghua University ² Independent ³ Peking University

Active Context Management

SCULPTOR: EMPOWERING LLMs WITH COGNITIVE AGENCY VIA ACTIVE

Mo Li¹, L.H. Xu², Qitai Tan¹, Lon
¹ Tsinghua University ² Independe

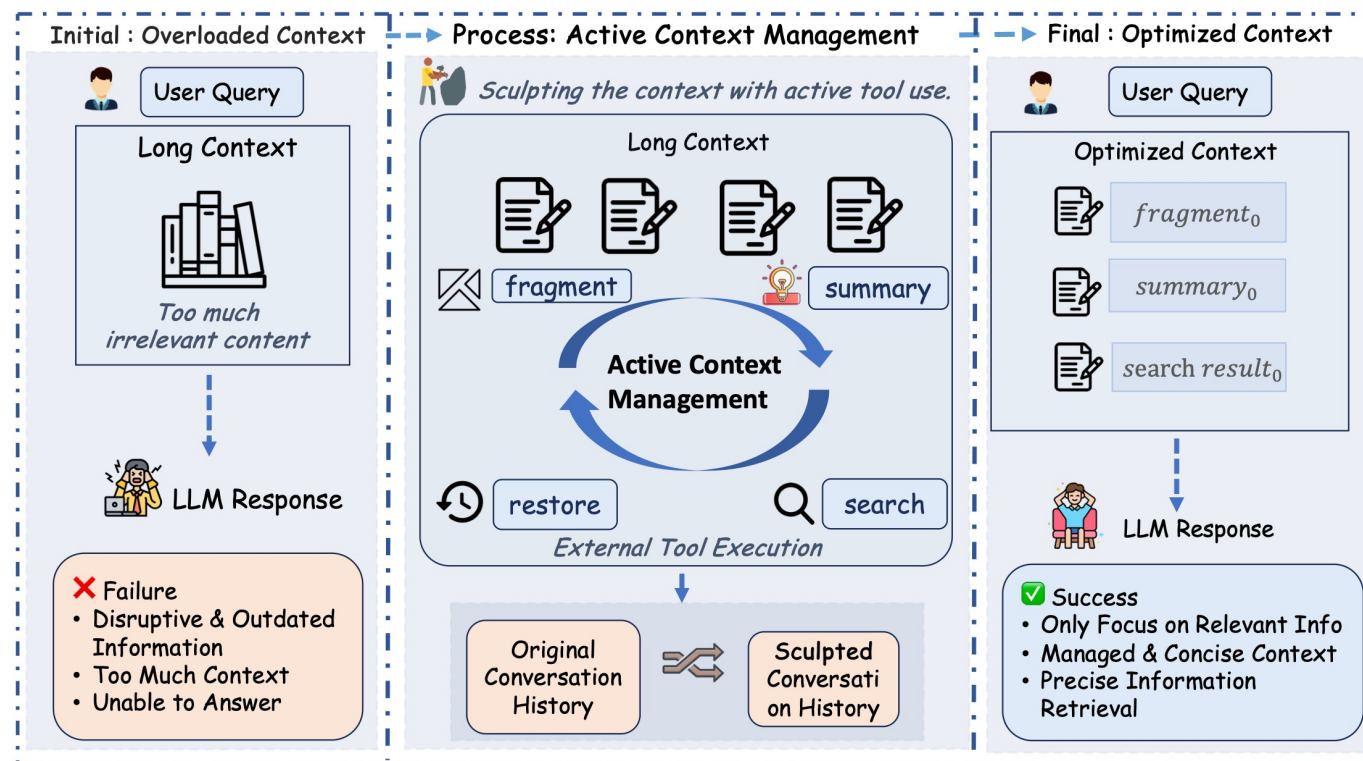
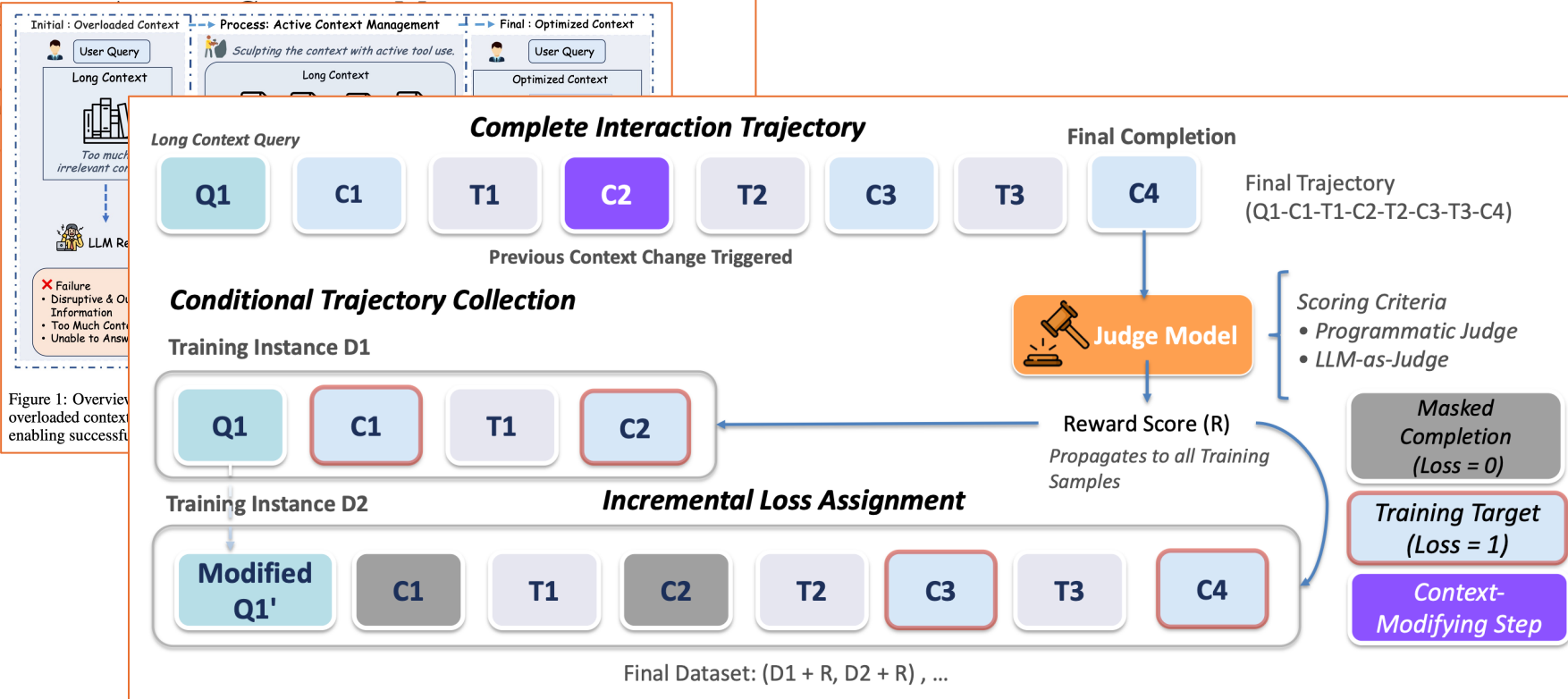


Figure 1: Overview of **SCULPTOR** framework: Through Active Context Management, LLMs transform overloaded contexts into optimized contexts using fragment, summary, search, and restore operations, enabling successful task completion where traditional approaches fail due to interference.

Active Context Management

SCULPTOR: EMPOWERING LLMs WITH COGNITIVE AGENCY

Mo Li¹, L
¹ Tsinghua



Active Context Management

SCULPTOR: EMPOWERING LLMs WITH COGNITIVE AGENCY

Mo Li¹, L
¹ Tsinghua

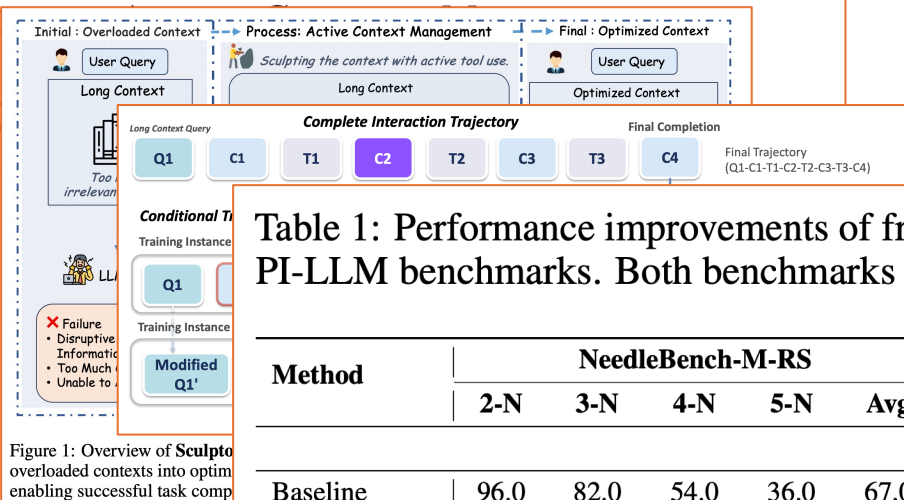


Figure 1: Overview of Sculptor: overloading contexts into optimized ones, enabling successful task completion.

Table 1: Performance improvements of frontier models with ACM Tools on NeedleBench-M-RS and PI-LLM benchmarks. Both benchmarks demonstrate substantial performance gains.

Method	NeedleBench-M-RS					PI-LLM (Update Count / Context Length)							
	2-N	3-N	4-N	5-N	Avg	4/1K	8/2K	16/4K	32/8K	64/16K	128/32K	256/64K	Avg
Claude-4-Sonnet													
Baseline	96.0	82.0	54.0	36.0	67.0	99.13	95.65	92.17	84.78	81.74	65.22	69.57	84.04
w/ ACM Tools	100.0	98.0	88.0	90.0	94.0	90.43	91.74	98.26	92.17	91.74	87.39	77.83	89.94
Δ	+4.0	+16.0	+34.0	+54.0	+27.0	-8.70	-3.91	+6.09	+7.39	+10.00	+22.17	+8.26	+5.90
GPT-4.1													
Baseline	90.0	64.0	30.0	8.0	48.0	96.96	91.30	79.57	67.83	63.04	63.91	50.43	73.29
w/ ACM Tools	96.0	84.0	60.0	44.0	71.0	92.17	89.13	93.04	83.91	76.09	64.35	60.43	79.87
Δ	+6.0	+20.0	+30.0	+36.0	+23.0	-4.79	-2.17	+13.47	+16.08	+13.05	+0.44	+10.00	+6.58
DeepSeek-V3													
Baseline	88.0	68.0	28.0	16.0	50.0	95.22	85.65	70.00	63.91	33.04	32.17	21.74	57.39
w/ ACM Tools	92.0	58.0	50.0	32.0	58.0	73.91	90.00	79.13	37.39	53.04	55.65	11.74	57.27
Δ	+4.0	-10.0	+22.0	+16.0	+8.0	-21.31	+4.35	+9.13	-26.52	+20.00	+23.48	-10.00	-0.12

The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management

**Tobias Lindenbauer^{1,2} Igor Slinko¹ Ludwig Felder²
Egor Bogomolov¹ Yaroslav Zharov¹**

¹JetBrains Research

²School of Computation, Information and Technology, Technical University of Munich
tobias.lindenbauer@jetbrains.com

The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for SE Agent Trajectory Management

Tobias Lindenbauer^{1,2} Igor Slinko
Egor Bogomolov¹ Yaroslav

¹JetBrains Research

²School of Computation, Information and Technology
tobias.lindenbauer@jetbrains.com

Token Type Distribution - Raw Agent

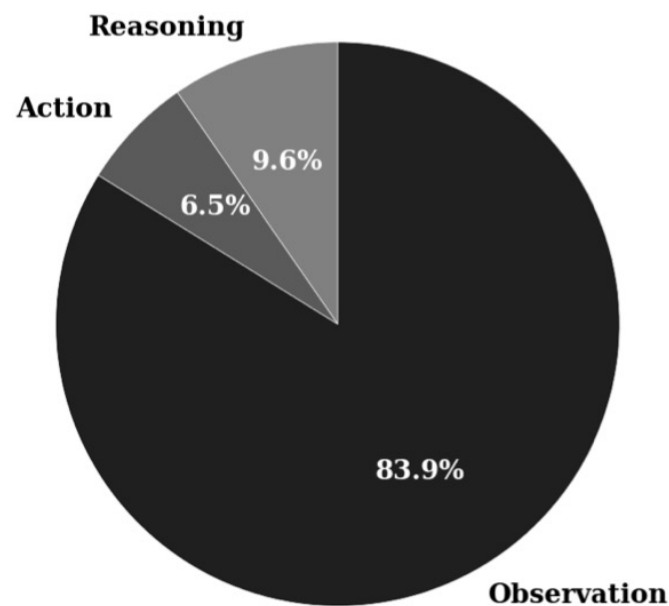


Figure 1: Environment observation tokens dominate the context window of an SE agent’s trajectory.

The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization

Token Type Distribution

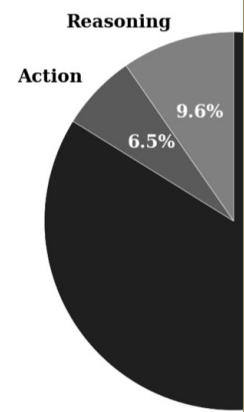


Figure 1: Environment tokens dominate the complexity of SWE agent’s trajectory

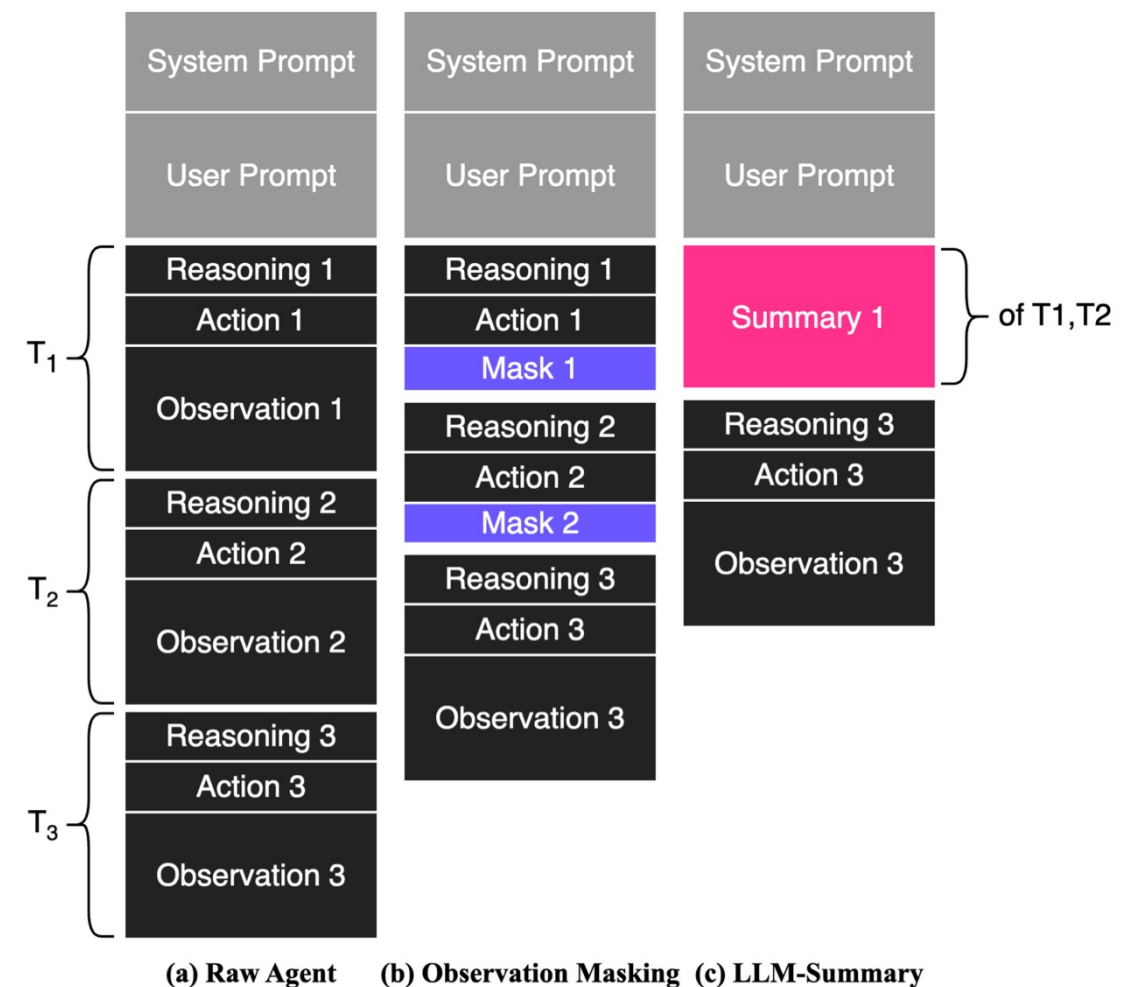


Figure 3: **Overview of the context management strategies evaluated in our work.** Box heights indicate the number of tokens in that portion of a typical trajectory. (a) The baseline ReAct-style [39] trajectory, where the context grows with each action-observation pair. (b) **LLM-based summarization** condenses older turns into a running summary and preserving a few recent turns in full (e.g., OpenHands [30]). (c) **Observation masking** replaces observations older than a fixed window of M turns (here, M=1) with a placeholder (e.g., SWE-agent [35]).

The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization

Token Type Distribution

Reasoning

Action

2^{Sc}

Figure 1: Tokens dominated by the SWE agent

Figure 3: Overview of the different styles [39] trajectories summarization in full (e.g., Context window of M

Table 1: **Comparison of context management strategies with 95% bootstrap confidence intervals.** We report change and significance ($\dagger$) compared to the *Raw Agent*. We report Solve Rate (effectiveness, $\uparrow$) and Instance Cost (efficiency, $\downarrow$). For each model, we **boldface** the best-performing context management strategy for each metric. All experiments use SWE-agent [35] on SWE-bench Verified [5]. Further details in Section C.

Model	Strategy	Solve Rate ($\%, \uparrow$)	Instance Cost ($\$, \downarrow$)
Qwen3-32B	Raw Agent	17.0 ± 3.3	1.12 ± 0.18
	Observation Masking	15.0 ± 3.1 (-11.8%)	0.55 ± 0.09 (-50.9%) $\dagger$
	LLM-Summary	16.0 ± 3.3 (-5.9%)	0.50 ± 0.07 (-55.4%) $\dagger$
Qwen3-32B (thinking)	Raw Agent	23.0 ± 3.7	0.51 ± 0.07
	Observation Masking	24.6 ± 3.8 (+7.0%)	0.46 ± 0.05 (-9.8%)
	LLM-Summary	24.8 ± 3.9 (+7.3%)	0.51 ± 0.06 (0.0%)
Qwen3-Coder 480B	Raw Agent	53.4 ± 4.3	1.29 ± 0.26
	Observation Masking	54.8 ± 4.4 (+2.6%)	0.61 ± 0.06 (-52.7%) $\dagger$
	LLM-Summary	53.8 ± 4.2 (+0.7%)	0.64 ± 0.06 (-50.4%) $\dagger$
Gemini 2.5 Flash	Raw Agent	32.8 ± 4.1	0.41 ± 0.08
	Observation Masking	35.6 ± 4.2 (+8.5%)	0.18 ± 0.03 (-56.1%) $\dagger$
	LLM-Summary	36.0 ± 4.1 (+9.8%)	0.24 ± 0.04 (-41.5%) $\dagger$
Gemini 2.5 Flash (thinking)	Raw Agent	40.4 ± 4.3	0.56 ± 0.10
	Observation Masking	36.4 ± 4.2 (-9.9%) $\dagger$	0.24 ± 0.04 (-57.1%) $\dagger$
	LLM-Summary	31.4 ± 4.0 (-22.3%) $\dagger$	0.25 ± 0.05 (-55.4%) $\dagger$



香港中文大學
The Chinese University of Hong Kong



SWE-MeM: Learning Adaptive Memory Management for Long-Horizon Coding Agents

Shuzheng Gao¹, Wenhao Zeng², Zhaojian Yu³, Jianqiao Wang⁴, Chaozheng Wang¹, Kai Cai⁴, Shilin He^{4,†}, Michael R. Lyu¹

¹The Chinese University of Hong Kong, China, ²Shanghai Jiao Tong University, China,

³Tsinghua University, China, ⁴ByteDance, China

[†]Corresponding author

SWE-M

Shuzhe

¹The C

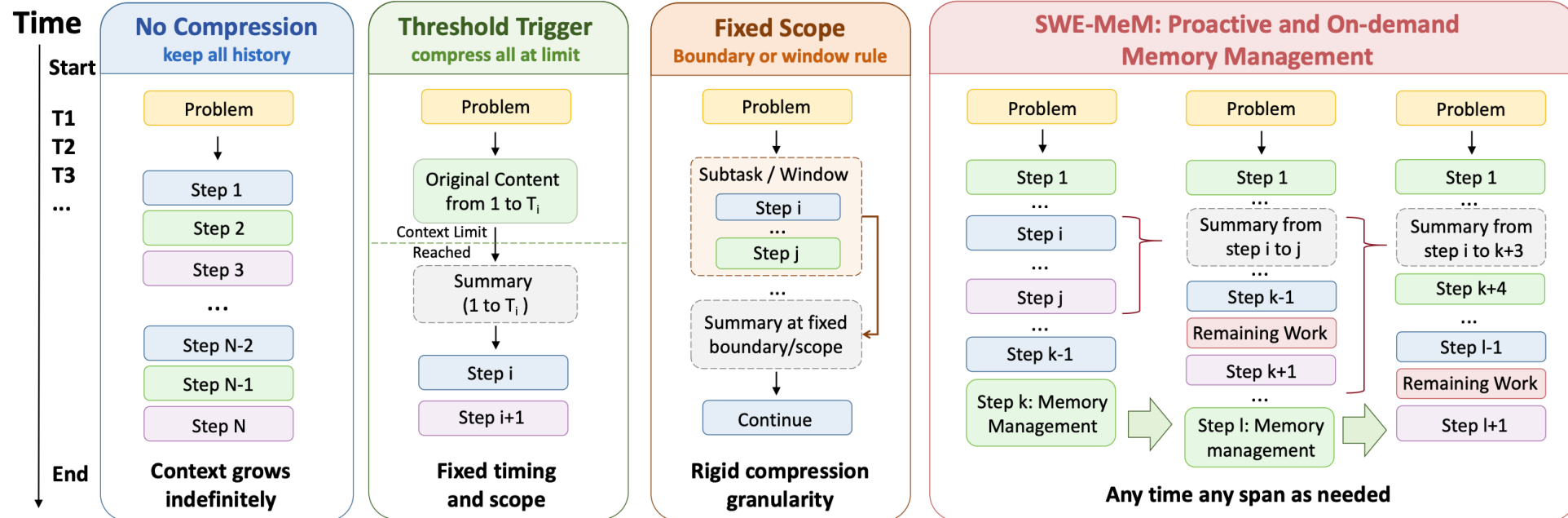


Figure 2 Comparison of different memory management methods. No compression mode causes the context to grow indefinitely; threshold-trigger methods compress only after the budget is reached; fixed-scope methods rely on rigid boundaries or windows; in contrast, SWE-MEM performs proactive and on-demand memory management over flexible context spans.

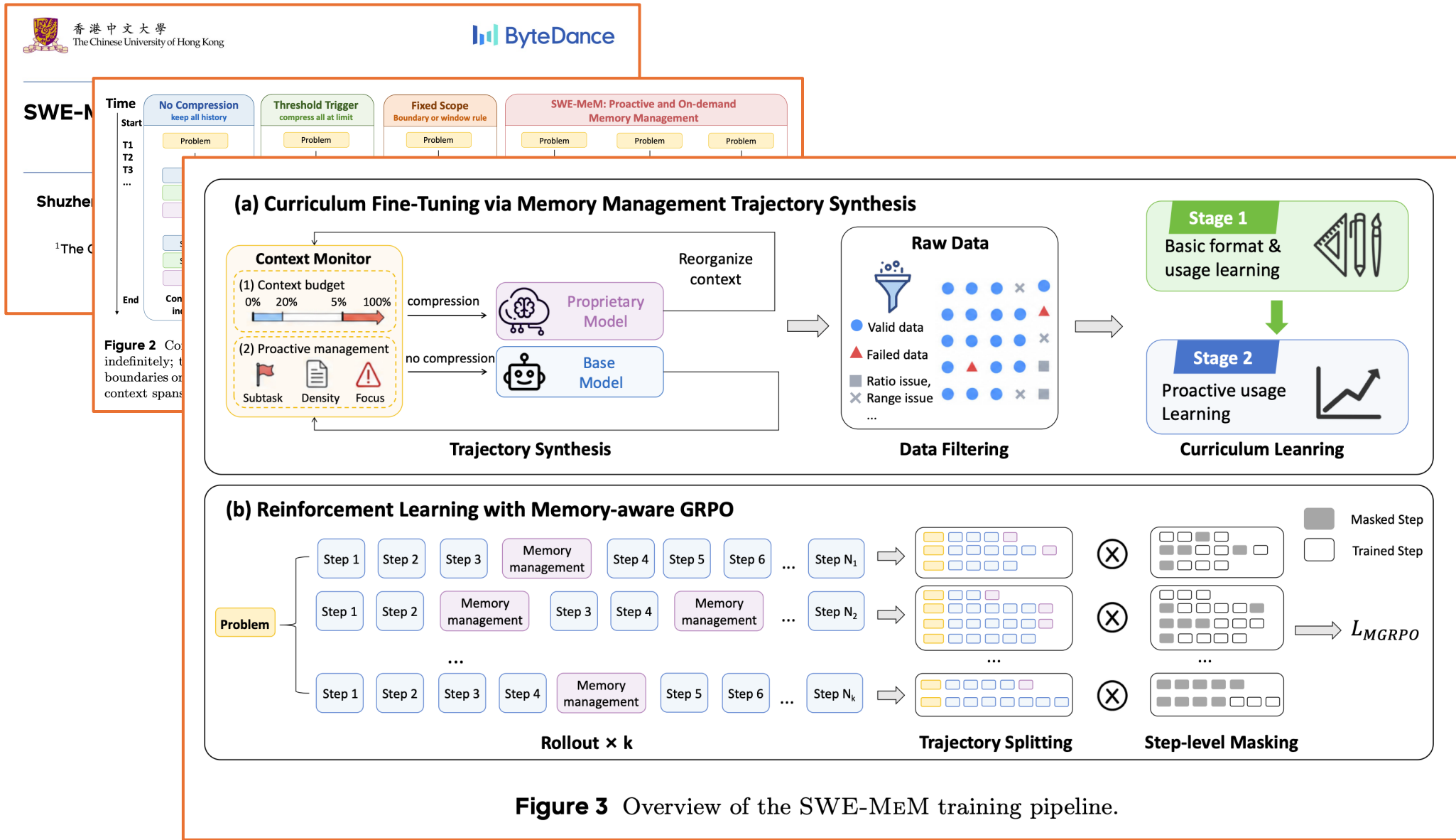


Figure 3 Overview of the SWE-MeM training pipeline.

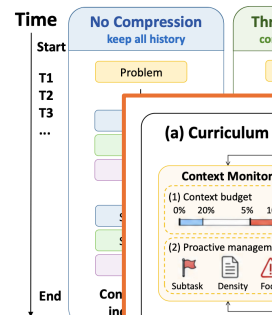


Figure 2 Context management strategies. (a) Curriculum learning: Context is managed indefinitely; boundaries on context spans. (b) Reinforcement learning: Context is managed by a monitor.

Method/Model	Base Model	Base Scaffold	Length Limitation	Resolve Rate
<10B Models				
ReAct Agent				
Qwen3-4B-Instruct [43]	—	OpenHands	128k	7.0
Qwen3-4B-Instruct [43]	—	OpenHands	32k	5.2
SWE-Gym-7B [29]	Qwen2.5-Coder-7B	OpenHands	32k	10.6
SWE-Mirror-7B [46]	Qwen2.5-Coder-7B	MOpenHands	128k	22.8
SWE-Dev-7B [45]	Qwen2.5-Coder-7B	OpenHands	160k	23.4
SWE-Master-4B [37]	Qwen3-4B-Instruct	OpenHands	128k	33.4
Klear-Agent-8B [47]	Qwen3-8B	Mini-SWE-agent-plus	64k	39.4
SWE-Lego-8B [42]	Qwen3-8B	OpenHands	128k	42.2
SWE-MeM				
SWE-MEM SFT	Qwen3-4B-Instruct	OpenHands	32k	41.6
SWE-MEM SFT+RL	Qwen3-4B-Instruct	OpenHands	32k	43.4
~30B Models				
ReAct Agent				
Qwen3-Coder-30B-A3B [43]	—	OpenHands	256k	51.6
Qwen3-Coder-30B-A3B [43]	—	OpenHands	32k	38.6
Seed-OSS-36B [3]	—	OpenHands	327K	55.2
SWE-Gym-32B [29]	Qwen2.5-Coder-32B	OpenHands	32k	20.6
SWE-Dev-32B [45]	Qwen2.5-Coder-32B	OpenHands	160k	36.6
DeepSWE-32B [44]	Qwen3-32B	R2E-Gym	128k	43.2
SWE-Mirror-32B [46]	Qwen2.5-Coder-32B	MOpenHands	128k	52.2
SWE-Lego-32B [42]	Qwen3-32B	OpenHands	128k	52.6
Summary Agent				
Threshold-Compression Agent [25]	Qwen2.5-Coder-32B	OpenHands	64k	53.8
Threshold-Compression Agent [41]	Seed-OSS-36B	OpenHands	32k	55.0
Folding Agent				
SWE-Compressor (150 step) [25]	Qwen2.5-Coder-32B	OpenHands	64k	54.8
SWE-Compressor [25]	Qwen2.5-Coder-32B	OpenHands	64k	57.8
Context Folding [41]	Seed-OSS-36B	OpenHands	32k	58.0
SWE-MeM				
SWE-MEM Workflow-only	Qwen3-Coder-30B-A3B	OpenHands	32k	58.4
SWE-MEM SFT (150 step)	Qwen3-Coder-30B-A3B	OpenHands	32k	57.2
SWE-MEM SFT	Qwen3-Coder-30B-A3B	OpenHands	32k	58.8
SWE-MEM SFT+RL	Qwen3-Coder-30B-A3B	OpenHands	32k	60.2

Table 1 Performance comparison on SWE-Bench Verified. SWE-MeM: Learning Adaptive Memory Management for Long-Horizon Coding Agents, Gao et al., 2026-06

Topics of Today

- Selected topics of agentic systems
 - ~~Tool selection problem~~
 - ~~Context management~~
 - Grammar-constrained decoding
 - Skills
 - Harness engineering

Review: Possible Next Token by Parser

Partial Program

```
min(input ++ [0])
```

Token Sequence

```
['min', '(', 'input', '++', '[', '0', '']
```

```
(Program) P <- L | N
(List) L <- 'input'
          | '[' ']'
          | '[' N ']'
          | L '++' L
(Number) N <- 'len' '(' L ')'
          | 'min' '(' L ')'
          | N '+' N
          | '0' | '1' | '2' | ...
```

Review: Possible Next Token by Parser

Partial Program

```
min(input ++ [0])
```

Token Sequence

```
['min', '(', 'input', '++', '[', '0', '']
```



Step 1

```
(Program) P <- L | N
(List) L <- 'input'
          | '[' ']'
          | '[' N ']'
          | L '++' L
(Number) N <- 'len' '(' L ')'
          | 'min' '(' L ')'
          | N '+' N
          | '0' | '1' | '2' | ...
```

Review: Possible Next Token by Parser

Partial Program

`min(input ++ [0])`

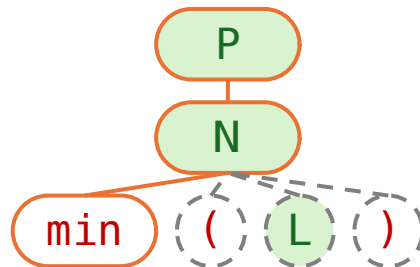
Token Sequence

`['min', '(', 'input', '++', '[', '0', ']']`



Step 1: Goal is P

`'min'` only appears in the rule `N <- 'min' '(' L ')'`



```
(Program) P <- L | N
(List) L <- 'input'
          | '[' ']'
          | '[' N ']'
          | L '++' L
(Number) N <- 'len' '(' L ')'
          | 'min' '(' L ')'
          | N '+' N
          | '0' | '1' | '2' | ...
```

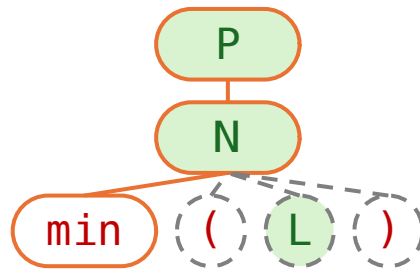
Review: Possible Next Token by Parser

Partial Program
`min(input ++ [0])`

Token Sequence
`['min', '(', 'input', '++', '[', '0', ']']`

Step 1: Goal is P

'min' only appears in the rule `N <- 'min' '(' L ')'`



The next token could only be ' ('

```
(Program) P <- L | N
(List) L <- 'input'
          | '[' ']'
          | '[' N ']'
          | L '++' L
(Number) N <- 'len' '(' L ')'
          | 'min' '(' L ')'
          | N '+' N
          | '0' | '1' | '2' | ...
```

Review: Possible Next Token by Parser

Partial Program

`min(input ++ [0])`

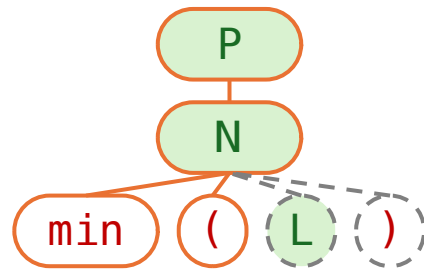
Token Sequence

`['min', '(', 'input', '++', '[', '0', ']']`



Step 2: Goal is P

`'min' '('` only appears in the rule `N <- 'min' '(' L ')'`



```
(Program) P <- L | N
(List) L <- 'input'
          | '[' ']'
          | '[' N ']'
          | L '++' L
(Number) N <- 'len' '(' L ')'
            | 'min' '(' L ')'
            | N '+' N
            | '0' | '1' | '2' | ...
```

Review: Possible Next Token by Parser

Partial Program

`min(input ++ [0])`

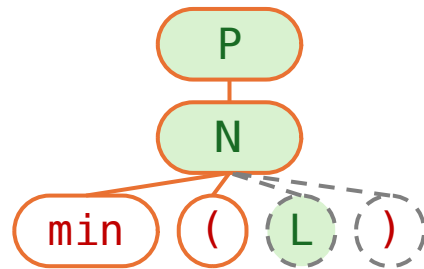
Token Sequence

`['min', '(', 'input', '++', '[', '0', ']']`



Step 2: Goal is P

`'min' '('` only appears in the rule `N <- 'min' '(' L ')'`



The next token could only be the first token expanded from L: `'input', '['`

```
(Program) P <- L | N
(List) L <- 'input'
          | '[' ']'
          | '[' N ']'
          | L '++' L
(Number) N <- 'len' '(' L ')'
            | 'min' '(' L ')'
            | N '+' N
            | '0' | '1' | '2' | ...
```

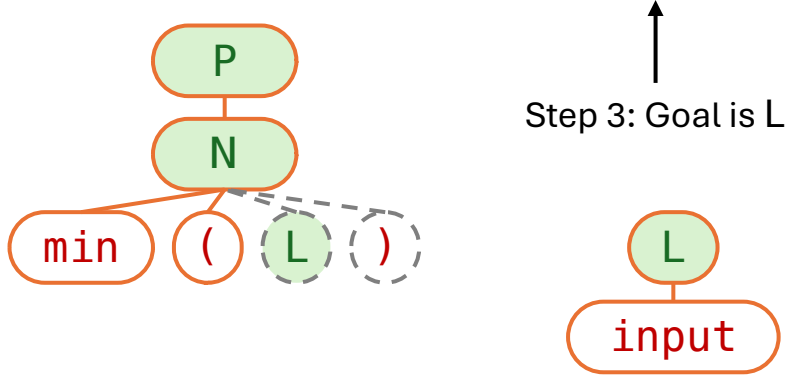
Review: Possible Next Token by Parser

Partial Program

`min(input ++ [0])`

Token Sequence

`['min', '(', 'input', '++', '[', '0', ']']`



```
(Program) P <- L | N
(List) L <- 'input'
          | '[' ']'
          | '[' N ']'
          | L '++' L
(Number) N <- 'len' '(' L ')'
            | 'min' '(' L ')'
            | N '+' N
            | '0' | '1' | '2' | ...
```

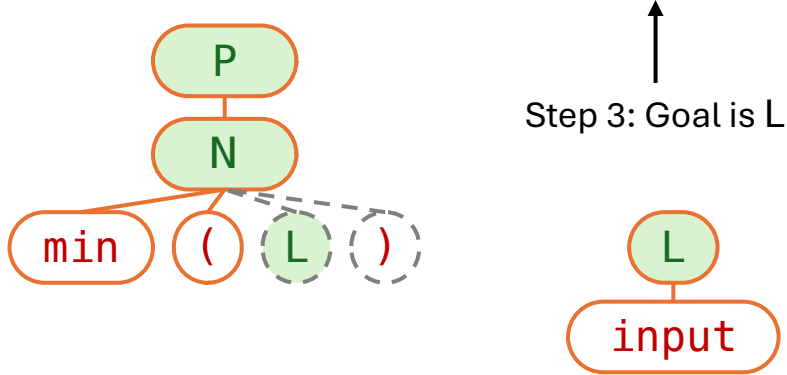
Review: Possible Next Token by Parser

Partial Program

`min(input ++ [0])`

Token Sequence

`['min', '(', 'input', '++', '[', '0', ']']`



```
(Program) P <- L | N
(List) L <- 'input'
          | '[' ']'
          | '[' N ']'
          | L '++' L
(Number) N <- 'len' '(' L ')'
            | 'min' '(' L ')'
            | N '+' N
            | '0' | '1' | '2' | ...
```

We can expand `L` into `input`, matching the third token

➔ Finishing `L`, next token is `)`

Review: Possible Next Token by Parser

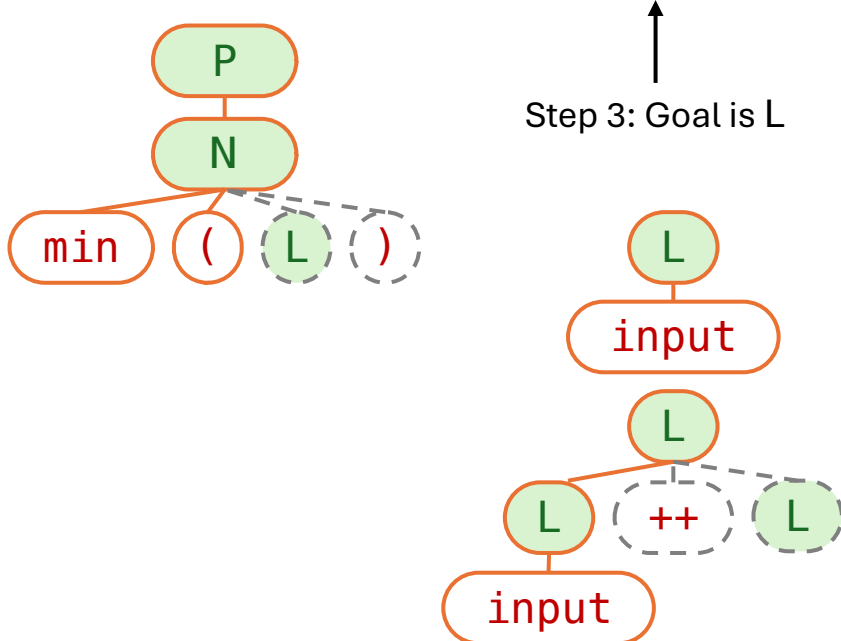
Partial Program

`min(input ++ [0])`

Token Sequence

`['min', '(', 'input', '++', '[', '0', ']']`

```
(Program) P <- L | N
(List) L <- 'input'
          | '[' ']'
          | '[' N ']'
          | L '++' L
(Number) N <- 'len' '(' L ')'
            | 'min' '(' L ')'
            | N '+' N
            | '0' | '1' | '2' | ...
```



We can expand **L** into **input**, matching the third token

➔ Finishing L, next token is **)**

We can expand **L** into **L + L**, where the first **L** can be expanded into **input**, matching the third token

Review: Possible Next Token by Parser

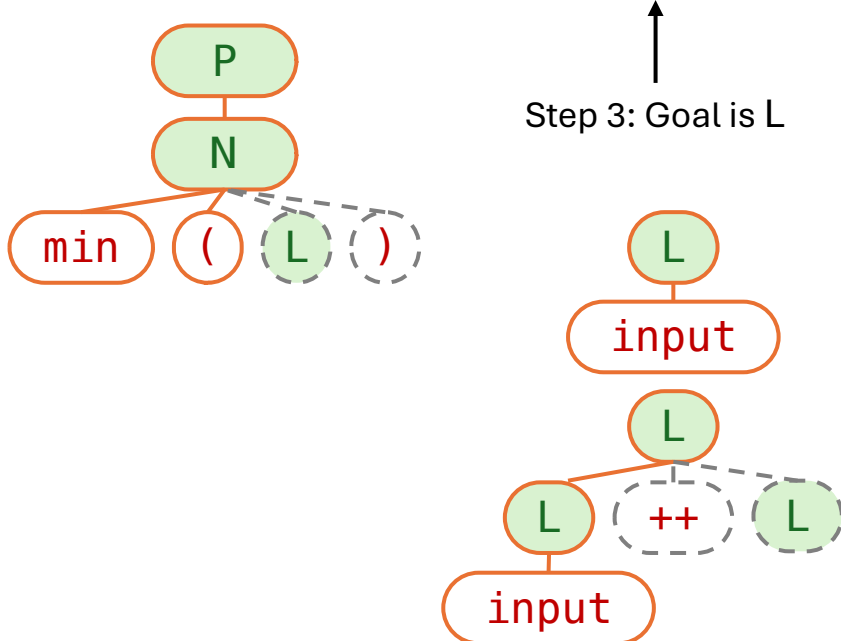
Partial Program

`min(input ++ [0])`

Token Sequence

`['min', '(', 'input', '++', '[', '0', ']']`

```
(Program) P <- L | N
(List) L <- 'input'
          | '[' ']'
          | '[' N ']'
          | L '++' L
(Number) N <- 'len' '(' L ')'
            | 'min' '(' L ')'
            | N '+' N
            | '0' | '1' | '2' | ...
```



We can expand **L** into **input**, matching the third token

➔ Finishing L, next token is **') '**

We can expand **L** into **L + L**, where the first **L** can be expanded into **input**, matching the third token

➔ Finishing the left L, next token is **' ++ '**

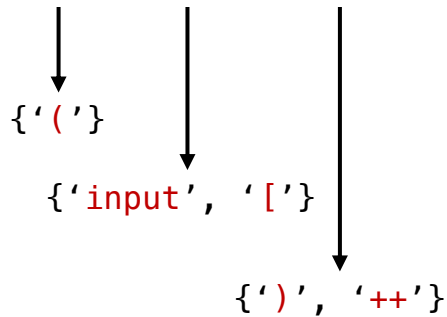
Review: Possible Next Token by Parser

Partial Program

```
min(input ++ [0])
```

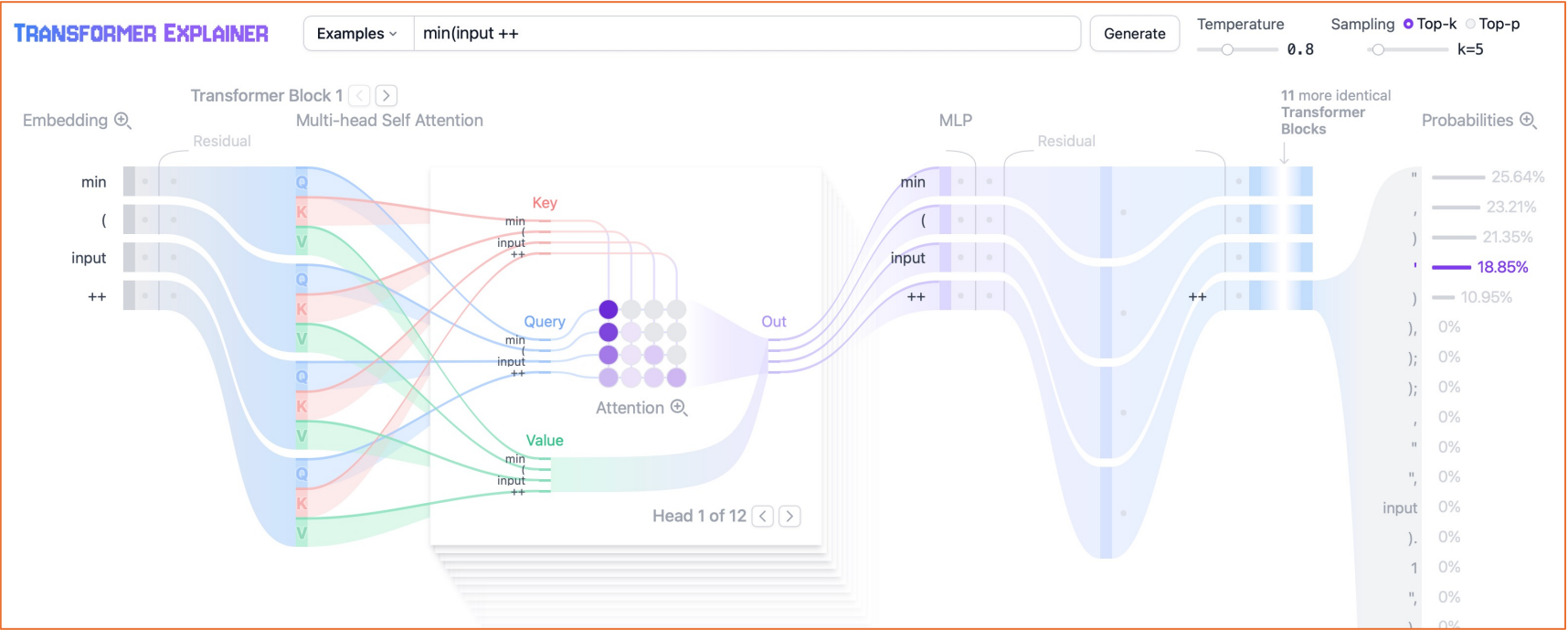
Token Sequence

```
['min', '(', 'input', '++', '[', '0', ']']
```



```
(Program) P <- L | N
(List) L <- 'input'
          | '[' ']'
          | '[' N ']'
          | L '++' L
(Number) N <- 'len' '(' L ')'
            | 'min' '(' L ')'
            | N '+' N
            | '0' | '1' | '2' | ...
```

Prefix: min(input ++



LLM Predicted
Next Token

Prefix Viability
Mask

input	0.01	1
)	0.21	0
'	0.19	0
"	0.26	0
;	0.01	0
[	0.02	1

	LLM Predicted Next Token	Prefix Viability Mask		
Prefix: <code>min(input ++</code>	<code>input</code> 0.01	1	×	=
	<code>)</code> 0.21	0		
	<code>'</code> 0.19	0		
	<code>"</code> 0.26	0		
	<code>;</code> 0.01	0		
	<code>[</code> 0.02	1		

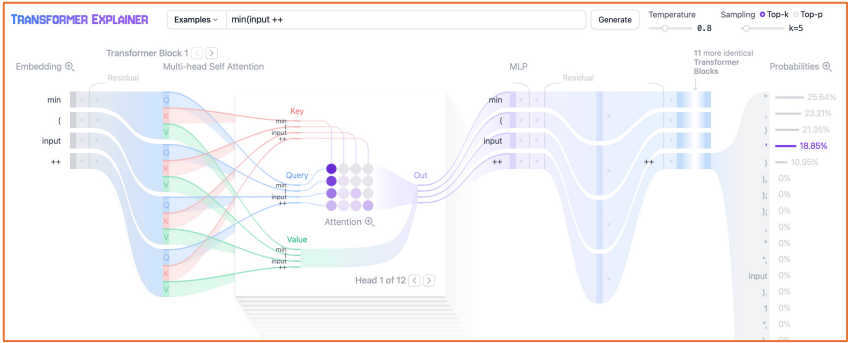
		LLM Predicted Next Token	Prefix Viability Mask	
Prefix: <code>min(input ++</code>	<code>input</code>	0.01	1	0.01
	<code>)</code>	0.21	0	0
	<code>'</code>	0.19	0	0
	<code>"</code>	0.26	0	0
	<code>;</code>	0.01	0	0
	<code>[</code>	0.02	1	0.02

× =

	LLM Predicted Next Token		Prefix Viability Mask			
Prefix: <code>min(input ++</code>	<code>input</code>	0.01	1	0.01		0.49
	<code>)</code>	0.21	0	-inf		0.0
	<code>'</code>	0.19	0	-inf		0.0
	<code>"</code>	0.26	0	-inf	softmax	0.0
	<code>;</code>	0.01	0	-inf		0.0
	<code>[</code>	0.02	1	0.02		0.51

Prefix: min(input ++

```
(Program) P <- L | N
(List) L <- 'input'
          | [' ']'
          | [' N ']'
          | L '++' L
(Number) N <- 'len' '(' L ')'
             | 'min' '(' L ')'
             | N '+' N
             | '0' | '1' | '2' | ...
```



LLM Predicted
Next Token

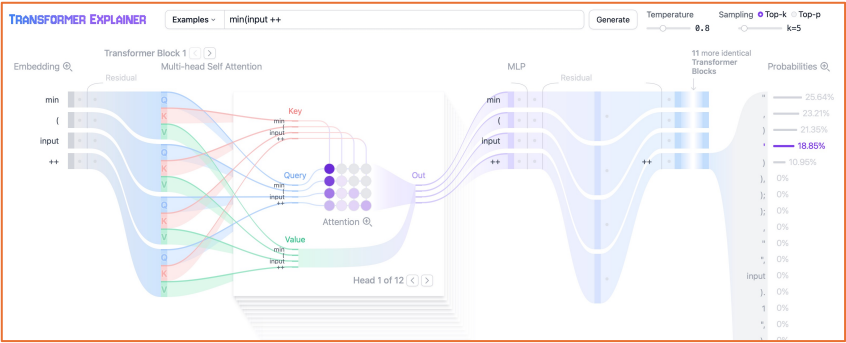
Prefix Viability
Mask

input	0.01	1	0.01	0.49
)	0.21	0	-inf	0.0
'	0.19	0	-inf	0.0
"	0.26	0	-inf	0.0
;	0.01	0	-inf	0.0
[	0.02	1	0.02	0.51

→ softmax →

Prefix: min(input ++

```
(Program) P <- L | N
(List) L <- 'input'
          | [' ']'
          | [' N ']'
          | L '++' L
(Number) N <- 'len' '(' L ')'
          | 'min' '(' L ')'
          | N '+' N
          | '0' | '1' | '2' | ...
```



LLM Predicted Next Token	Prefix Viability Mask			
input	0.01	1	0.01	0.49
)	0.21	0	-inf	0.0
'	0.19	0	-inf	0.0
"	0.26	0	-inf	0.0
;	0.01	0	-inf	0.0
[	0.02	1	0.02	0.51

→ softmax →

XGrammar-2: Dynamic and Efficient Structured Generation Engine for Agentic LLMs

Linzhang Li*

blemiade_qinchuan@sjtu.edu.cn
Shanghai Jiao Tong University
China

Ziyi Xu

xzy2022@sjtu.edu.cn
Shanghai Jiao Tong University
China

Yixin Dong*[†]

yixind@andrew.cmu.edu
Carnegie Mellon University
USA

Alexander Jiang

akj2@andrew.cmu.edu
Carnegie Mellon University
USA

Guanjie Wang

irfnfnkemed@sjtu.edu.cn
Shanghai Jiao Tong University
China

Tianqi Chen[†]

tqchen@cmu.edu
Carnegie Mellon University, NVIDIA
USA

XGrammar-2: Dynamic and Efficient Structured Generation Engine for Agentic LLMs

Linzhang Li*
blemiade_qinchuan@sjtu.edu.cn
Shanghai Jiao Tong University
China

Ziyi Xu
xzy2022@sjtu.edu.cn
Shanghai Jiao Tong University
China

Tool Calling:
Llama Tool Calling
(defined by request's
tool set)

Response Protocol:
OpenAI Harmony
Response Format

Generation Structure

```
<function=get_weather>  
{"city": ".*"}</function>  
  
<function=search>  
{"url": ".*"}</function>
```

```
<|channel|>analysis  
<|message|>".*"  
  
<|channel|>commentary  
to=functions.get_weather  
<|message|>{"city": ".*"}
```

Output of LLM

OK, I need to call a function.
<function=get_weather> {"city":
"Shanghai"}

```
<|channel|>analysis  
<|message|>OK, I need to think  
about it.  
<|channel|>commentary  
to=fucntions.get_weather  
<|message|>{"city": "Shanghai"}
```



Tag



Dispatched Structure



Free-form Text

XGrammar-2: Dynamic and Efficient Structured Generation Engine for Agentic LLMs

Generation Structure

Output of LLM

<function=get_weather>
{"city": "Shanghai"}</function>

OK, I need to call a function.

g
du.cn
iversity

blemiade
Shangh

xzy
Shangh

Response Protoc
OpenAI Harmony
Response Format

Tag

Model Name	Type	Correct Call Rate	Correct Schema Rate
Llama-3.2-1B	w/o XGrammar-2	6.07%	22.07%
	w/ XGrammar-2	32.84%	100.00%
Llama-3.2-3B	w/o XGrammar-2	33.12%	40.70%
	w/ XGrammar-2	77.75%	100.00%
Llama-3.1-8B	w/o XGrammar-2	59.48%	66.95%
	w/ XGrammar-2	80.93%	100.00%
Llama-3.1-70B	w/o XGrammar-2	45.60%	51.94%
	w/ XGrammar-2	86.41%	100.00%

Table 6: The function calling accuracy rate and the JSON schema validity rate.

Strict tool use

 Copy page 

Enforce JSON Schema compliance on Claude's tool inputs with **grammar-constrained sampling**.

Setting `strict: true` on a tool definition guarantees Claude's tool inputs match your JSON Schema by constraining the model's token sampling to schema-valid outputs (a technique called grammar-constrained sampling). This page covers why strict mode matters for agents, how to enable it, and common use cases. For the supported JSON Schema subset, see [JSON Schema limitations](#). For non-strict schema guidance, see [Define tools](#).

Strict tool use validates tool parameters, ensuring Claude calls your functions with correctly-typed arguments. Use strict tool use when you need to:

- Validate tool parameters
- Build agentic workflows
- Ensure type-safe function calls
- Handle complex tools with nested properties

Strict tool use

[Copy page](#)

Enforce JSON Schema compliance on Claude's tool inputs with grammar-constrained sampling.

Setting `strict: true` in the model's token sampling parameters covers why strict mode is a subset, see [JSON Schema](#).

Strict tool use validates tool inputs and enforces strict tool use when you use the tool.

- Validate tool parameters
- Build agentic workflows
- Ensure type-safe tool inputs
- Handle complex tool inputs

Quick start

[cURL](#) [CLI](#) **[Python](#)** [TypeScript](#) [C#](#) [Go](#) [Java](#) [PHP](#) [Ruby](#)



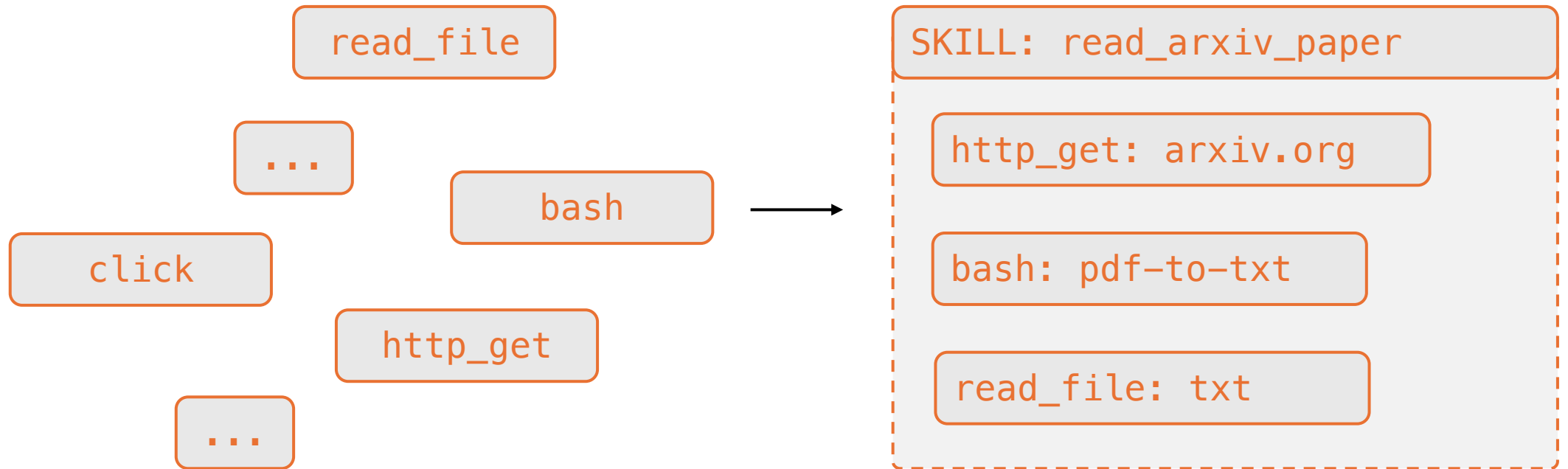
```
client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-opus-5-5",
    max_tokens=1024,
    messages=[{"role": "user", "content": "What's the weather like in San Francisco?"}],
    tools=[
        {
            "name": "get_weather",
            "description": "Get the current weather in a given location",
            "strict": True, # Enable strict mode
            "input_schema": {
                "type": "object",
                "properties": {
                    "location": {
                        "type": "string",
                        "description": "The city and state, e.g. San Francisco, CA",
                    },
                },
            },
            "unit": {
```

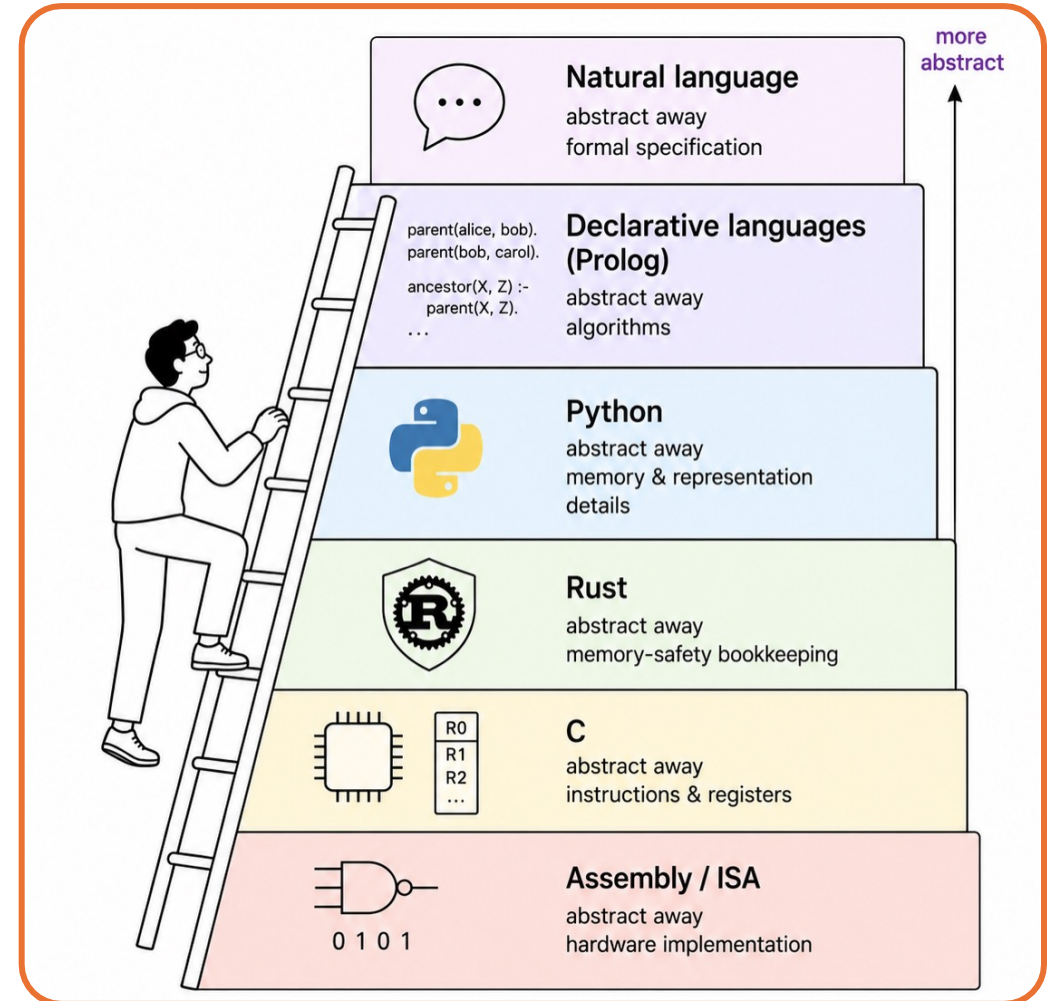
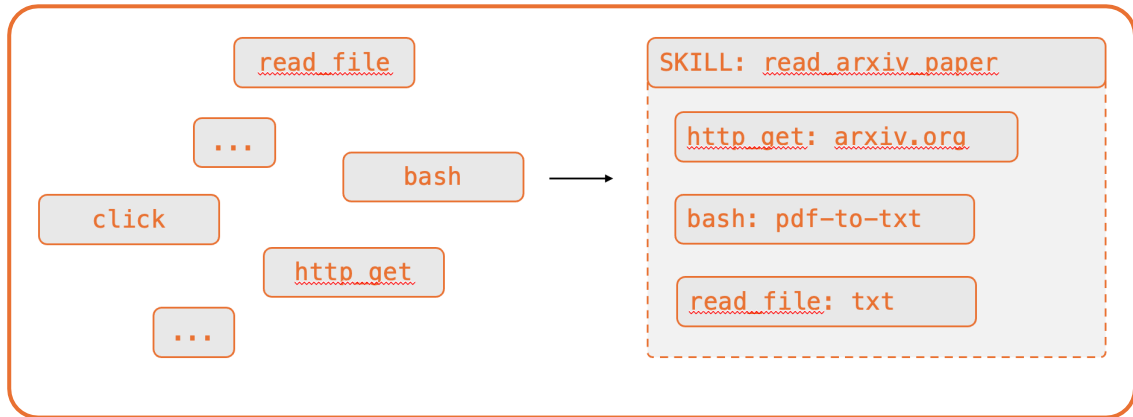
Topics of Today

- Selected topics of agentic systems
 - ~~Tool selection problem~~
 - ~~Context management~~
 - ~~Grammar-constrained decoding~~
 - Skills
 - Harness engineering

Skills as Another Layer of Abstraction

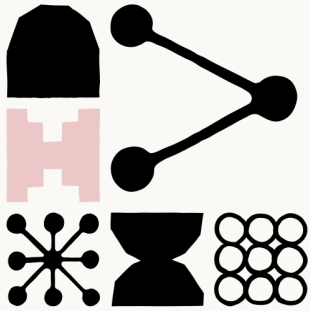


Skills as Another Layer of Abstraction



Skills

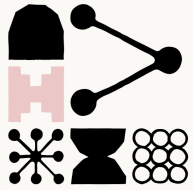
Engineering at Anthropic



Equipping agents for the real world with Agent Skills

Published Oct 16, 2025

Claude is powerful, but real work requires procedural knowledge and organizational context. Introducing Agent Skills, a new way to build specialized agents using files and folders.



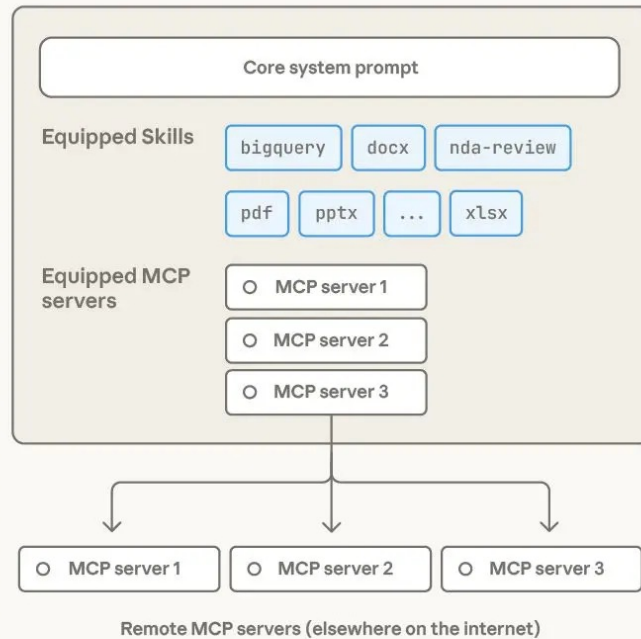
Equipping agents for the real world with Agent Skills

Published Oct 16, 2025

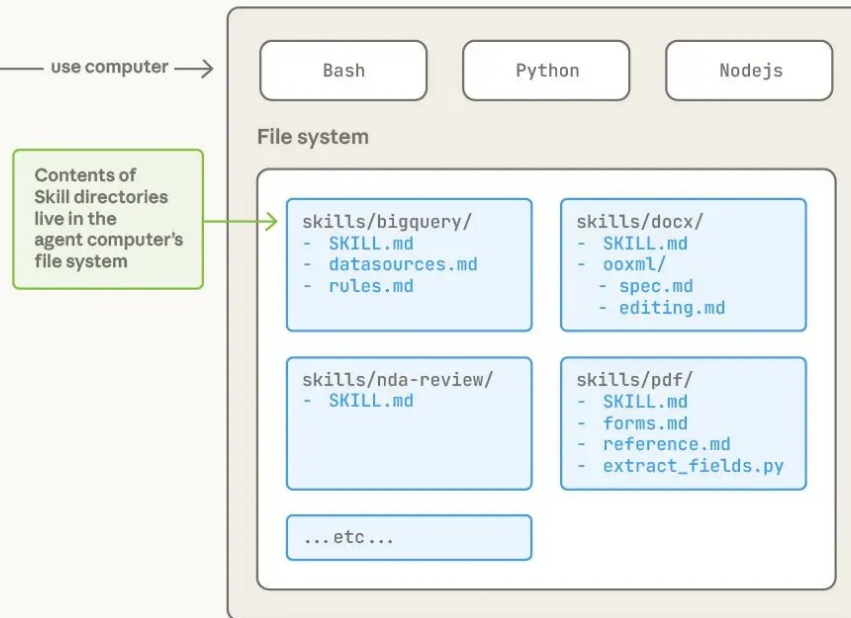
Claude is p
context. Int
and folders

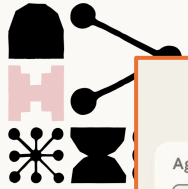
Agent + Skills + Virtual Machine

Agent configuration



Agent virtual machine





Equipping agents for the real world

Published Oct 16, 2023

Agent + Skills + Virtual Machine

Agent configuration

Agent virtual machine

Equipped Skills

Equipped MCP servers

O MCP server

Remove

A simple SKILL.md file

pdf/SKILL.md

YAML Frontmatter

```
---
name: pdf
description: Comprehensive PDF toolkit for extracting text and tables,
merging/splitting documents, and filling-out forms.
---
```

Markdown

Overview

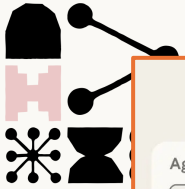
This guide covers essential PDF processing operations using Python libraries and command-line tools. For advanced features, JavaScript libraries, and detailed examples, see `./reference.md`. If you need to fill out a PDF form, read `./forms.md` and follow its instructions.

Quick Start

```
```python
from pypdf import PdfReader, PdfWriter
```

```
Read a PDF
reader = PdfReader("document.pdf")
print(f"Pages: {len(reader.pages)}")
```

```
...
```



## Equipping agents for the real world

Published Oct 16, 2023

### Agent + Skills + Virtual Machine

#### Agent configuration

Equipped Skill

Equipped MCP servers

MCP server

Remove

## Skills and the Context Window

### Context Window

Agent's System Prompt

bigquery

docx

nda-review

pdf

pptx

...

xlsx

Short snippets from each Skill appended to the system prompt

Initial message from user

User: Fill out this PDF based on what you know about me  
Attached: /mnt/uploads/order\_form.pdf

Claude: Certainly! I'll use the PDF Skill to help

Claude decides to "trigger" the PDF Skill by reading it

Tool use: Bash("cat /mnt/skills/pdf/SKILL.md")

Tool result: {stdout: "...contents of the SKILL.md file..."}

Tool use: Bash("cat /mnt/skills/pdf/forms.md")

Claude decides to follow the reference from the SKILL.md to the forms.md file

Tool result: {stdout: "...contents of the forms.md file..."}

Published as a conference paper at COLM 2025

---

## Inducing Programmatic Skills for Agentic Tasks

**Zora Zhiruo Wang   Apurva Gandhi   Graham Neubig   Daniel Fried**  
Carnegie Mellon University  
{zhiruow,apurvag,gneubig,dfried}@cs.cmu.edu

## Inducing Programmatic Skills

Zora Zhiruo Wang Apurva Gandhi Gra  
Carnegie Mellon University  
{zhiruow, apurvag, gneubig, dfried}@cs.cmu.

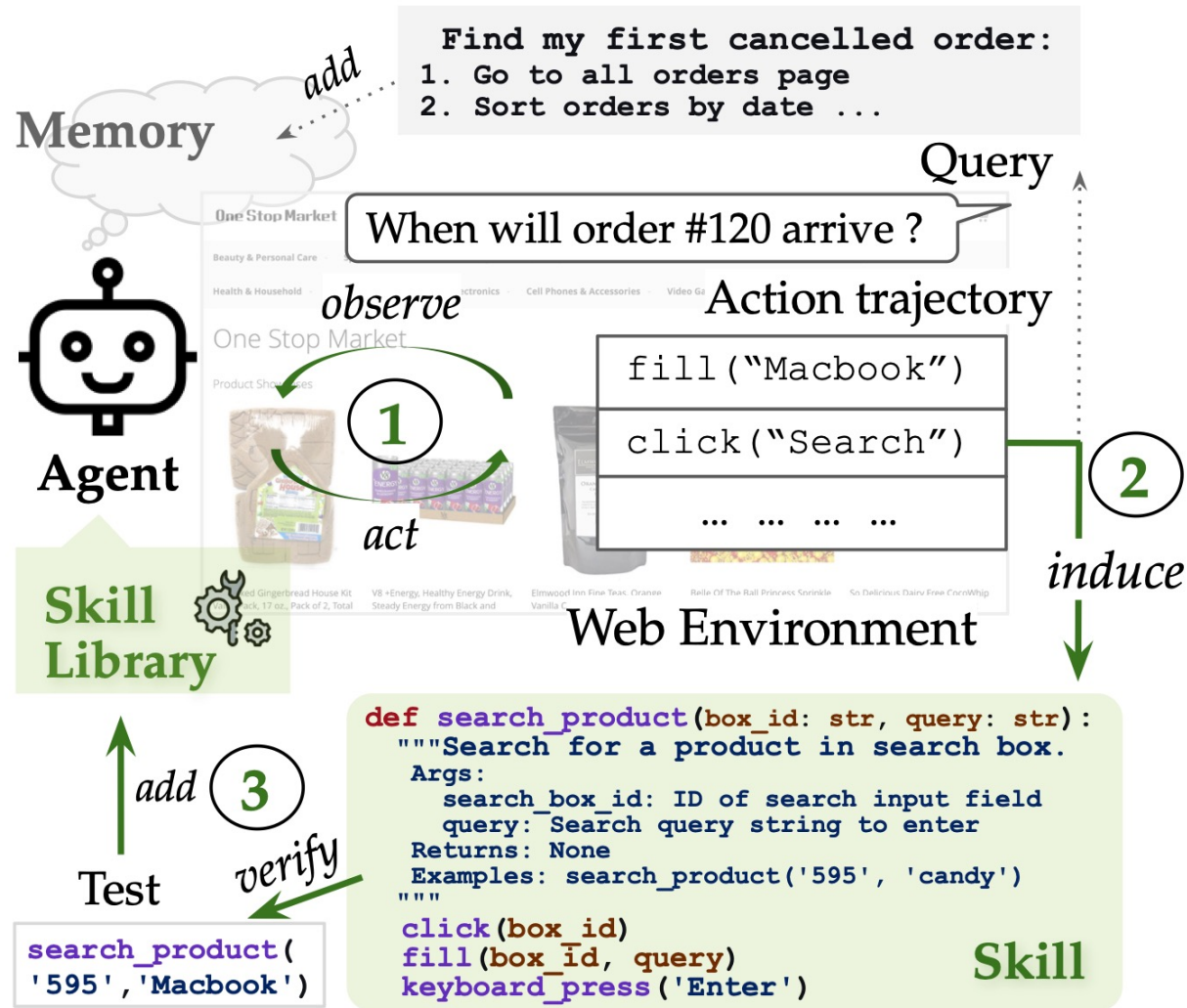
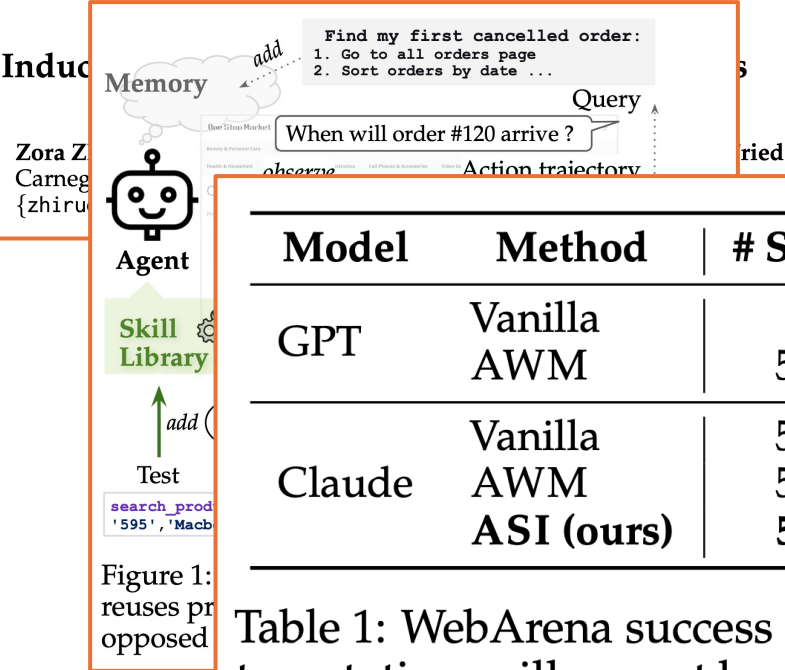


Figure 1: Online adaptive agent that induces and reuses programmatic skills as actions (bottom), as opposed to adding textual skills in memory (top).





Model	Method	# Steps	SR	Shop	Admin	Reddit	GitLab	Maps	Multi
GPT	Vanilla	-	12.3	13.9	10.4	6.6	15.0	15.6	8.3
	AWM	5.9	35.5	32.1	29.1	<b>54.7</b>	<b>35.0</b>	42.2	18.8
Claude	Vanilla	5.6	32.7	32.6	36.8	36.8	26.1	38.5	<b>20.8</b>
	AWM	5.9	36.3	34.8	39.0	51.9	28.9	39.4	18.8
	<b>ASI (ours)</b>	<b>5.0</b>	<b>40.4</b>	<b>40.1</b>	<b>44.0</b>	<b>54.7</b>	32.2	<b>43.1</b>	<b>20.8</b>

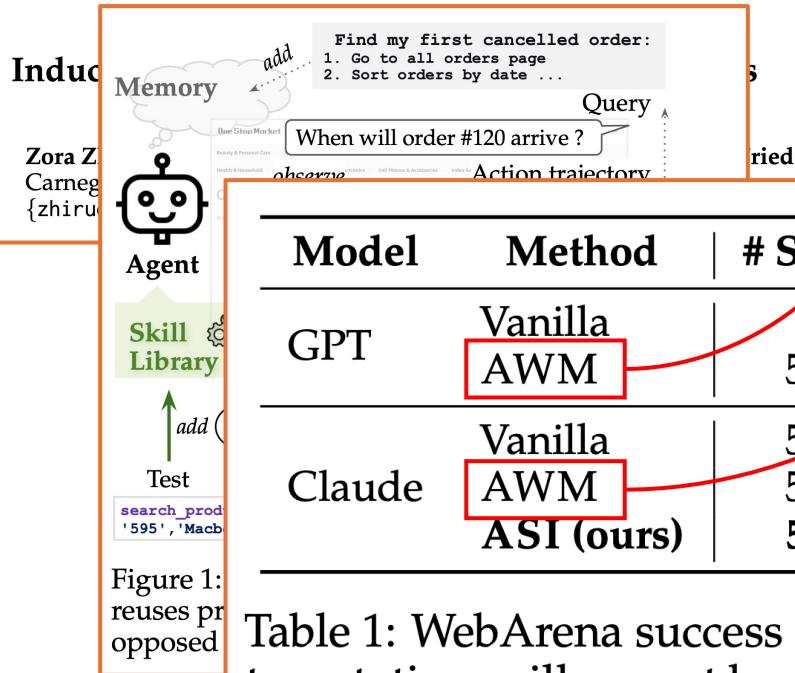
Table 1: WebArena success rate by adaptive agents with programmatic skills, in comparison to a static vanilla agent baseline, and a text-skill learning adaptive agent.



# AGENT WORKFLOW MEMORY

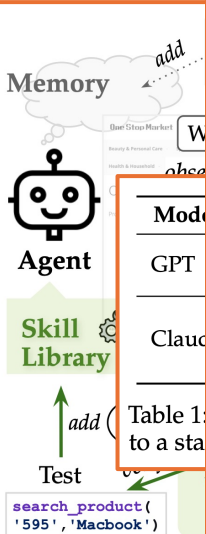
Zora Zhiruo Wang<sup>ℳ</sup> Jiayuan Mao<sup>ℳ</sup> Daniel Fried<sup>ℳ</sup> Graham Neubig<sup>ℳ</sup>  
<sup>ℳ</sup> Carnegie Mellon University <sup>ℳ</sup> Massachusetts Institute of Technology

Published as a conference paper at COLM 2025



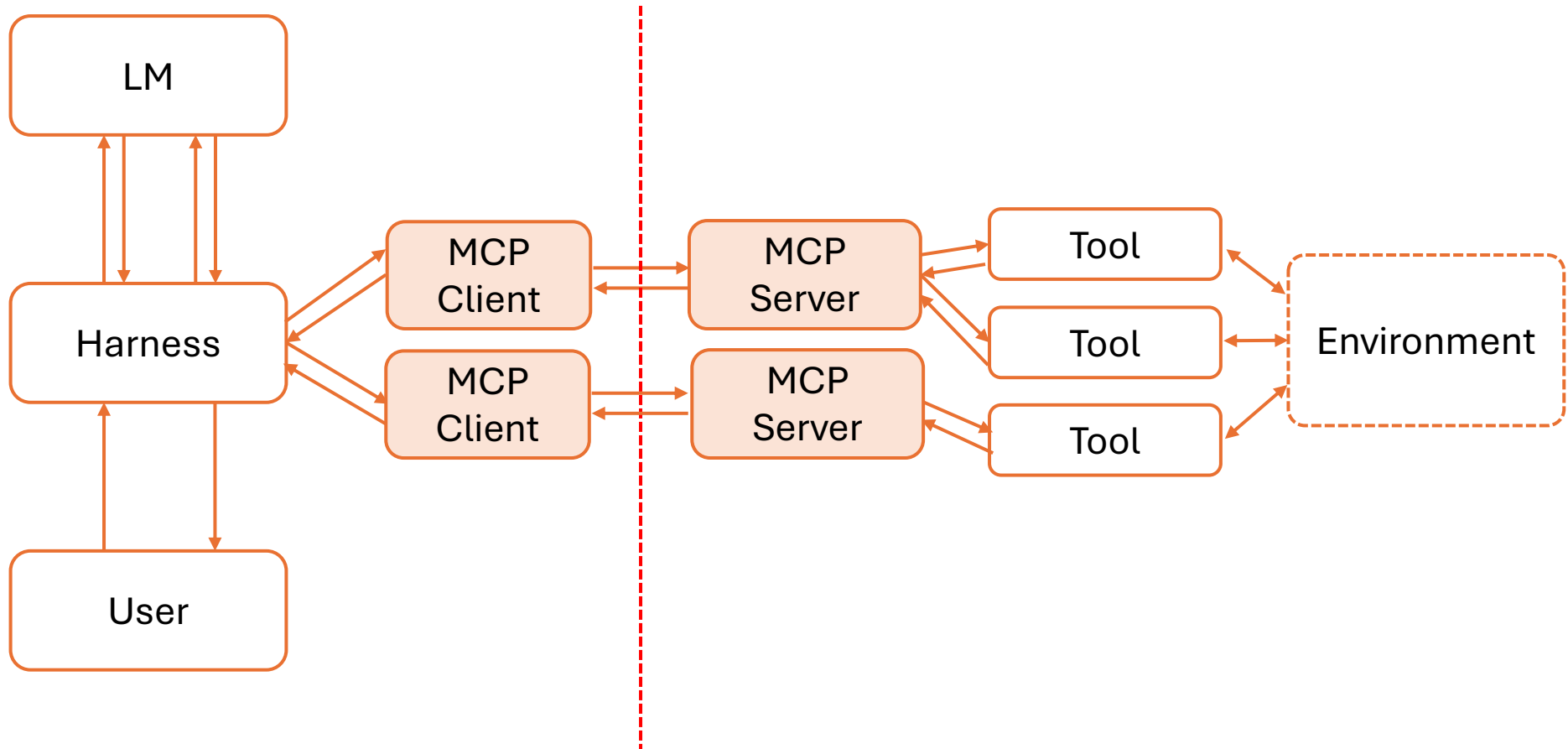
Model	Method	# Steps	SR	Shop	Admin	Reddit	GitLab	Maps	Multi
GPT	Vanilla	-	12.3	13.9	10.4	6.6	15.0	15.6	8.3
	AWM	5.9	35.5	32.1	29.1	54.7	35.0	42.2	18.8
Claude	Vanilla	5.6	32.7	32.6	36.8	36.8	26.1	38.5	20.8
	AWM	5.9	36.3	34.8	39.0	51.9	28.9	39.4	18.8
	ASI (ours)	5.0	40.4	40.1	44.0	54.7	32.2	43.1	20.8

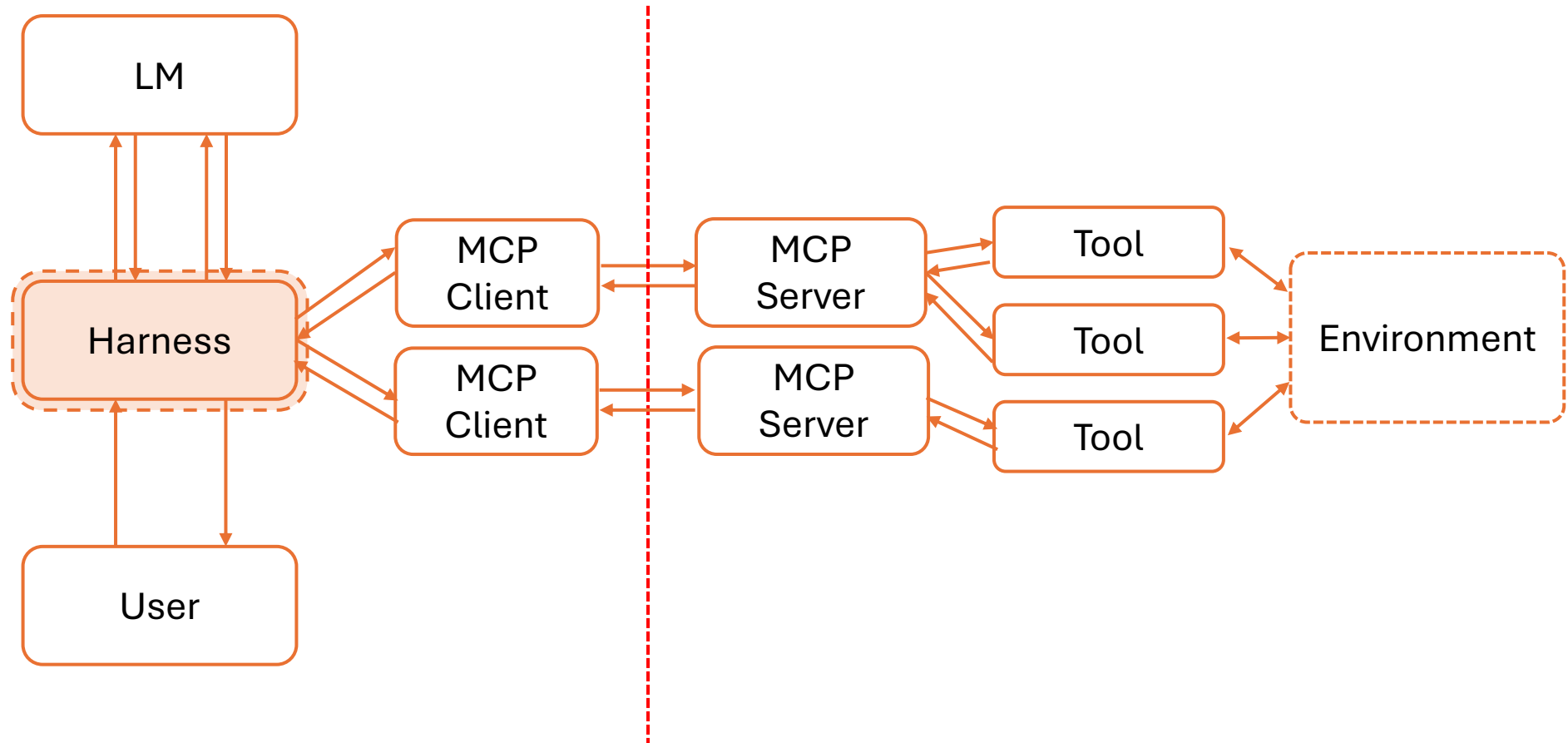
Table 1: WebArena success rate by adaptive agents with programmatic skills, in comparison to a static vanilla agent baseline, and a text-skill learning adaptive agent.

Published as a conference paper at COLM 2025   Induc   Zora Z Carneg {zhiru    Figure 1: Online reuses program opposed to addi	<table><thead><tr><th>Programmatic Skills</th><th>Textual Skills</th></tr></thead><tbody><tr><td><pre>def search_product(search_box_id: str, query: str):     """Search for a product using the search box.     Args:         search_box_id: ID of the search input field         query: Search query string to enter     Returns:         None      Examples:         search_product('595', 'sony bluetooth headphones')         """     click(search_box_id)     fill(search_box_id, query)     keyboard_press('Enter')</pre></td><td> Task: Search for a product and add it to wish list Action Trajectory: ```fill(621, {product_name})``` # Enter the product name in the search box ```click(478)``` # Click the search button to execute the search ```click({product_link})``` # Click the product to check more details ```click(1769)``` # Click the "Add to Wish List" link    Task: Search for a product's price range in the store Action Trajectory: ```fill(565, {product_name})``` # Enter the product name in the search box ```click(570)``` # Click the search button to execute the search ```noop(1000)``` # Wait for search results to load ```send_msg_to_user({price_range_info})``` # Analyze and report the price range findings from the search results    Task: Search for gaming accessories within a date range Action Trajectory: ```click(1274)``` # Navigate to the Video Games category ```fill(473, {search_terms})``` # Enter search terms including product name and year ```click(478)``` # Execute the search </td></tr></tbody></table>  Table 2: Example textual and program skills induced on the shopping website.	Programmatic Skills	Textual Skills	<pre>def search_product(search_box_id: str, query: str):     """Search for a product using the search box.     Args:         search_box_id: ID of the search input field         query: Search query string to enter     Returns:         None      Examples:         search_product('595', 'sony bluetooth headphones')         """     click(search_box_id)     fill(search_box_id, query)     keyboard_press('Enter')</pre>	Task: Search for a product and add it to wish list Action Trajectory: ```fill(621, {product_name})``` # Enter the product name in the search box ```click(478)``` # Click the search button to execute the search ```click({product_link})``` # Click the product to check more details ```click(1769)``` # Click the "Add to Wish List" link      Task: Search for a product's price range in the store Action Trajectory: ```fill(565, {product_name})``` # Enter the product name in the search box ```click(570)``` # Click the search button to execute the search ```noop(1000)``` # Wait for search results to load ```send_msg_to_user({price_range_info})``` # Analyze and report the price range findings from the search results      Task: Search for gaming accessories within a date range Action Trajectory: ```click(1274)``` # Navigate to the Video Games category ```fill(473, {search_terms})``` # Enter search terms including product name and year ```click(478)``` # Execute the search
Programmatic Skills	Textual Skills				
<pre>def search_product(search_box_id: str, query: str):     """Search for a product using the search box.     Args:         search_box_id: ID of the search input field         query: Search query string to enter     Returns:         None      Examples:         search_product('595', 'sony bluetooth headphones')         """     click(search_box_id)     fill(search_box_id, query)     keyboard_press('Enter')</pre>	Task: Search for a product and add it to wish list Action Trajectory: ```fill(621, {product_name})``` # Enter the product name in the search box ```click(478)``` # Click the search button to execute the search ```click({product_link})``` # Click the product to check more details ```click(1769)``` # Click the "Add to Wish List" link      Task: Search for a product's price range in the store Action Trajectory: ```fill(565, {product_name})``` # Enter the product name in the search box ```click(570)``` # Click the search button to execute the search ```noop(1000)``` # Wait for search results to load ```send_msg_to_user({price_range_info})``` # Analyze and report the price range findings from the search results      Task: Search for gaming accessories within a date range Action Trajectory: ```click(1274)``` # Navigate to the Video Games category ```fill(473, {search_terms})``` # Enter search terms including product name and year ```click(478)``` # Execute the search				

# Topics of Today

- Selected topics of agentic systems
  - ~~Tool selection problem~~
  - ~~Context management~~
  - ~~Grammar-constrained decoding~~
  - ~~Skills~~
  - Harness engineering





# What is a Harness



# The design space of a Harness

- **Iterative Loop**
  - When to stop? Are there special branches for planning, execution, interaction, retry/repair when error?
- **Action Space / Output Control**
  - What is the tool calling protocol? How to manage tools? How to control tool calls' outputs?
- **Context Management**
  - How to control agent's memory and context? When to compact?
- **Orchestration (Sub-agents, Multi-Agent Systems, etc.)**
  - ...

# **Harness Engineering: Anatomy, Architecture, and Evolution of Coding Agents**

A Source-Code Study of Eleven Systems

Paul Barbaste<sup>\*1,2</sup>, Tristan Darrigol<sup>2</sup>, Germain Vu<sup>2</sup>, and Tom Wiltberger<sup>2</sup>

<sup>1</sup>Inclusive Brains

<sup>2</sup>Wavestone AI Lab

July 2026



Ha  
Anatomy, Archi  
A Source  
Paul Barbaste\*<sup>1,2</sup>, Trista

Table 3: System selection criteria. The top eleven rows are the study corpus; OMNIGENT is analyzed as a meta-harness contrast point only.

System	Provider	Language	Version	Selection Rationale
OPENHANDS	OpenHands (ex-All-Hands-AI)	Python	V1 SDK v1.34.0 (Jul. 2026)	Mature modular framework
AIDER	Community (P. Gauthier)	Python	v0.86.3.dev (May 2026; maintenance mode)	Polymorphic edit strategies
CLAUDE CODE	Anthropic	TypeScript	source snapshot Mar. 2026 (binary 2.1.206, Jul. 2026)	Anthropic’s flagship agent
CODEX	OpenAI	Rust	rust-v0.144.1 (Jul. 2026)	OpenAI’s flagship agent
GEMINI CLI	Google	TypeScript	v0.50.0 (Jul. 2026)	Google’s flagship agent
MISTRAL VIBE	Mistral AI	Python	v2.19.1 (Jul. 2026)	Mistral’s flagship agent
MINI-SWE-AGENT	Princeton/Stanford	Python	v2.4.5 (Jul. 2026)	Minimalist research baseline
HERMES	Nous Research	Python	0.18.2 (rel. 2026.7.7.2, Jul. 2026)	Self-improving skills; hybrid personal/SWE harness
PI	earendil-works (M. Zechner)	TypeScript	v0.80.6 (Jul. 2026)	Minimal core, maximal extensibility
OPENCODE	Anomaly (ex-SST)	TypeScript	v1.17.18 (Jul. 2026)	Client/server architecture; most-starred dedicated coding agent
OPENCRAW	Community	TypeScript	v2026.6.11 (Jul. 2026)	Contrast: multi-channel assistant
OMNIGENT	Databricks	Python	v0.4.0 (Jul. 2026)	Contrast: meta-harness above harnesses

Anat

Paul

Harn

Table 3: System selection criteria as a meta-harness contrast point	
System	Provider
OPENHANDS	OpenHands
AIDER	All-Hands-Community (P. Gauthier)
CLAUDE CODE	Anthropic
CODEx	OpenAI
GEMINI CLI	Google
MISTRAL VIBE	Mistral AI
MINI-SWE-AGENT	Princeton/Stanford
HERMES	Nous Research
PI	OpenAI (M. Zechner)
OPeNCoDe	Anomaly (ex-SST)
OPeNCLAW	Community
OMNIGENT	Databricks

Table 4: High-level comparison of the eleven systems, with each system’s signature capability. CLAUDE CODE cells here and in the other July-dated tables reflect the March-2026 source snapshot, the newest source available to us; the shipping binary (2.1.206, July 2026) may differ.										
System	Lang.	Scale	Provider	Tools	Multi-ag.	Sandbox	UI	Signature		
OPENHANDS	Python	Medium	Multi	25+	Yes	Docker/ Apptainer/ remote	Web+CLI +SDK	Resource-locked parallel tools; hosts rival harnesses as ACP backends		
AIDER	Python	Small	Multi	13 fmt	No	None	CLI	13 polymorphic edit formats; the corpus’s only ranked repo map		
CLAUDE CODE	TS	Large	Anthropic	43	Yes	Opt-in runtime + worktree	CLI	Deferred tool loading; prompt-cache-sharing forks; the reference others copy		
CODEx	Rust	Very lg.	OpenAI	25–30	Yes	Native (3 OS)	CLI/TUI	Agent-maintained cross-session memory; tool calls as V8-executed code		
GEMINI CLI	TS	Very lg.	Google	35+	Yes	Native (3 OS)	CLI/TUI	Model routing as runtime scheduling; the corpus’s only A2A server		
MISTRAL VIBE	Python	Small	Mistral+	12+	Yes	Worktree opt.	CLI/TUI	Middleware-pipeline loop; rewind exposed over ACP		
MINI-SWE-AGENT	Python	Tiny	Multi	1	No	Docker+	CLI	The 100-line floor: all seven subsystems, minimally		
HERMES	Python	Very lg.	Multi (owned)	69	Yes	6 pluggable backends	CLI/TUI/ Web+28 ch.	Self-improving skill loop; lineage compaction; verify-on-stop		
PI	TS	Medium	Multi, 35 prov.	7	Ext.	None (by design)	CLI/TUI	Everything-is-an-extension core; session-tree version control		
OPeNCoDe	TS	Large	Multi	17	Yes	None (policy)	Client/ server	Client/server harness; syntax-aware command permissioning		
OPeNCLAW	TS	Large	Multi	109+	Yes	None	Multi-ch.	Gateway delegation; Active-Memory sub-agent before each reply		

Anat

Paul

# Harness Engineering:

ing

Table 3: System selection criteria. The top eleven rows are the study corpus; OMNIGENT is analyzed as a meta-harness contrast point only.

System	Provider	Language	Version	Selection Rationale						
OPENHANDS	Table 4: High-level comparison of the eleven systems, with each system’s signature capability. CLAUDE CODE cells here and in the other July-dated tables reflect the March-2026 source snapshot, the newest source available to us; the shipping binary (2.1.206, July 2026) may differ.									
AIDER										
CLAUDE CODE	System	Lang.	Scale	Provider	Tools	Multi-ag.	Sandbox	UI	Signature	
	OPENHANDS	Python	Medium	Multi	25+	Yes	Docker/Apptainer/	Web+CLI+SDK	Resource-locked tools; hosts rival hardware	
CODEx	AIDER									
GEMINI CLI	CLAUDE CODE									
MISTRAL VIBE	CLAUDE CODE									
MINI-SWE-AG	CLAUDE CODE									
HERMES	CODEx									
PI	GEMINI CLI									
OPeNCoDE	MISTRAL VIBE									
OPeNCLAW	MINI-SWE-AG									
OMNIGENT	HERMES	Python	Very lg.	Multi (owned)	69	Yes	6 pluggable backends	CLI/TUI/Web+28 ch.	Self-improving skill loop; lineage compaction; verify-on-stop	
	PI	TS	Medium	Multi, 35 prov.	7	Ext.	None (by design)	CLI/TUI	Everything-is-an-extension core; session-tree version control	
	OPeNCoDE	TS	Large	Multi	17	Yes	None (policy)	Client/server	Client/server harness; syntax-aware command permissioning	
	OPeNCLAW	TS	Large	Multi	109+	Yes	None	Multi-ch.	Gateway delegation; Active-Memory sub-agent before each reply	

## 7 LLM Integration and Model–Agent Co-design

How an agent scaffold relates to its underlying LLM is one of the most consequential design decisions in the system. The corpus spans the full range from tight single-provider coupling to complete provider agnosticism, and the position on that spectrum has direct implications for the optimizations available to the scaffold.

# Logistics – Week 5

- Assignment 2

Assignment 2 - Evaluating Coding LLMs

SEP 25, 2026 6:08 AM

OCT 9, 2026 11:59 PM

Late Due Date: OCT 12, 2026 11:59 PM

4

100%

ON

- Due in ~one week (Oct 9)
  - In case your LLM API key is running out of credit, please register for your own key following our tutorial!
  - Otherwise please
- Final Project